

Creating the Wide View Filter’s Weights from a Polynomial Fit

My interpretation of the mathematical construction of the Wide View Filter.

1 Introduction: The Savitzky-Golay Principle

At its core, this method is a Savitzky–Golay derivative estimator. Savitzky and Golay’s original 1964 contribution was to show that one can fit a polynomial locally by least squares and recover derivatives directly from the fitted coefficients [1]. Our derivation uses the same principle, but adapts it to two-dimensional image neighborhoods.

Although a polynomial fit is usually described as solving for unknown coefficients, the least-squares solution is linear in the sampled pixel values. As a result, any fitted coefficient, including a derivative coefficient, can be expressed as one fixed weighted sum of the input pixels. The resulting weighted sum for each pixel is effectively the filter weights, similar to how Sobel or Prewitt weights are defined.

2 A Pedagogical Example: The 3x3 Square Neighborhood

2.1 Defining the Image Patch and Polynomial Model

We begin with a 3×3 example because it makes the algebra visible. Conceptually, nothing new happens here since this is the same two-dimensional polynomial-fitting procedure developed in the Savitzky–Golay tradition and later extended to image neighborhoods by Gorry, Meer et al., and Luo et al. [2], [3], [4] in the ‘90s and early ‘00s. The small patch is used primarily to save me a lot of algebra and to make the logic more transparent. The same principles apply to larger patches, and we will show how to generalize to those later.

2.1.1 The Local Patch and Polynomial Ansatz

As a simplification, let us begin with a local 3×3 image patch. We denote this neighborhood by $\mathbf{B}$:

$$\mathbf{B} = \begin{bmatrix} I_{-1,1} & I_{0,1} & I_{1,1} \\ I_{-1,0} & I_{0,0} & I_{1,0} \\ I_{-1,-1} & I_{0,-1} & I_{1,-1} \end{bmatrix}. \quad (2.1)$$

This matrix contains the pixel intensities in a local 3×3 window, with the pixel of interest located at the center. Now suppose we aim to fit a degree-1 polynomial to the matrix $\mathbf{B}$. One simple way to represent that local polynomial is

$$p(x, y) = c_0 + c_1x + c_2y. \quad (2.2)$$

Here c_0 represents the local baseline brightness, ideally close to the brightness of the center pixel. The coefficients c_1 and c_2 describe how the brightness changes in the x and y directions, respectively. Since an edge can be understood as a spatial change in brightness, these change terms are the quantities we care about most for edge detection. In this sense, c_1 and c_2 are the derivative coefficients, while c_0 mainly helps the polynomial match the local patch.

If we instead use a degree-2 polynomial, we could write

$$p(x, y) = c_0 + c_1x + c_2y + c_3x^2 + c_4xy + c_5y^2. \quad (2.3)$$

In this case, c_0 , c_1 , and c_2 play the same roles as before, while the additional coefficients capture curvature in the local intensity surface. Intuitively, they describe how the rate of change itself varies across the patch. Even when the polynomial degree is increased, the main coefficients used for estimating the local directional change are still the first-derivative terms. Increasing the degree

simply gives the fit more flexibility, which can improve accuracy in some cases when the local image structure is not well described by a purely linear model.

With all that mathematical kerfuffle out of the way, the only question that remains is how to solve for those unknown coefficients. To do that, we write one fitting equation for each pixel in the local patch and then collect them into a single linear system.

2.1.2 From Pixel Coordinates to Fitting Equations

For example, if we want to write the formula for the degree-1 polynomial, we begin with Equation 2.2 and substitute the pixel coordinates into it. The pixel at $(-1, -1)$ has coordinates $x = -1$ and $y = -1$, so we plug those into the polynomial and plug in the known coordinates of a pixel in the patch.

For the pixel at $(-1, -1)$, this gives

$$p(-1, -1) = c_0 + c_1(-1) + c_2(-1) \quad (2.4)$$

$$= c_0 - c_1 - c_2. \quad (2.5)$$

Since this fitted value should match the observed pixel intensity at that location, and since the degree-1 polynomial is only an approximation to the local pixel intensities, we use $\approx$ instead of exact equality in the fitting equations.

$$c_0 - c_1 - c_2 \approx I_{-1,-1}. \quad (2.6)$$

We then repeat this same substitution for the other pixel coordinates in the 3x3 patch. Doing so yields

$$c_0 - c_1 - c_2 \approx I_{-1,-1} \quad (2.7)$$

$$c_0 - c_2 \approx I_{0,-1} \quad (2.8)$$

$$c_0 + c_1 - c_2 \approx I_{1,-1} \quad (2.9)$$

$$c_0 - c_1 \approx I_{-1,0} \quad (2.10)$$

$$c_0 \approx I_{0,0} \quad (2.11)$$

$$c_0 + c_1 \approx I_{1,0} \quad (2.12)$$

$$c_0 - c_1 + c_2 \approx I_{-1,1} \quad (2.13)$$

$$c_0 + c_2 \approx I_{0,1} \quad (2.14)$$

$$c_0 + c_1 + c_2 \approx I_{1,1}. \quad (2.15)$$

The resulting set of equations is all one needs to solve for the system. However, as you may remember from primary school, a system of equations can often be written more efficiently in matrix form. Doing so makes the mathematics easier to organize and manipulate. To see how this becomes matrix form, consider the first equation:

$$c_0 - c_1 - c_2 \approx I_{-1,-1}. \quad (2.16)$$

This can be rewritten in a matrix form as

$$[1, -1, -1] \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} \approx I_{-1,-1}, \quad (2.17)$$

where the first value of the first matrix remains 1, since it is the coefficient of c_0 , and the second and third values are -1 because they are the coefficients of c_1 and c_2 , respectively. The row vector $[1, -1, -1]$ is called a row of the design matrix, and the column vector of unknowns is called the coefficient vector. The right-hand side is just the observed pixel intensity at that location.

The same idea applies to every other pixel in the patch. For example, the pixel at $(0, -1)$ gives the row equation $[1, 0, -1]z \approx I_{0,-1}$, and the pixel at $(1, -1)$ gives $[1, 1, -1]z \approx I_{1,-1}$. Once all of these row equations are stacked together, they form one compact matrix system

$$\mathbf{A}z \approx \mathbf{b}. \quad (2.18)$$

where $\mathbf{A}$ is the design matrix containing the coefficients of the unknowns for each pixel, $\mathbf{z}$ is the vector of unknown polynomial coefficients, and $\mathbf{b}$ is the vector of observed pixel intensities.

2.2 Constructing the Matrix System

2.2.1 Vectorizing the Pixel Data

To derive $\mathbf{b}$ and $\mathbf{A}$, we first need to decide how to stack the pixel values from the patch into a vector. One common way is to read the pixels in row-major order, starting from the top-left corner and moving left to right, then down to the next row. Doing so gives us

$$\mathbf{b} = [I_{-1,-1}, I_{0,-1}, I_{1,-1}, I_{-1,0}, I_{0,0}, I_{1,0}, I_{-1,1}, I_{0,1}, I_{1,1}]^\top. \quad (2.19)$$

2.2.2 Building the Design Matrix

Then, the design matrix is just the basis $[1, x, y]$ evaluated at one sample location. Therefore

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & -1 & 0 & 1 & -1 & 0 & 1 \\ -1 & -1 & -1 & 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}^\top. \quad (2.20)$$

Then, finally, to solve for the unknown coefficients, we use the least-squares solution to the system in Equation 2.18.

2.3 The Least-Squares Solution

2.3.1 The Least-Squares Objective

At this point, we have nine approximate equations but only three unknown coefficients. In general, there is no reason to expect one vector $\mathbf{z}$ to satisfy every equation exactly. So instead of solving $\mathbf{A}z = \mathbf{b}$ exactly, we choose the coefficient vector that makes the overall mismatch as small as possible in the least-squares sense:

$$\hat{\mathbf{z}} = \arg \min_{\mathbf{z}} \|\mathbf{A}z - \mathbf{b}\|^2. \quad (2.21)$$

To make that step explicit, let's write out the objective function that we are minimizing, with E representing error

$$E(\mathbf{z}) = \|\mathbf{A}z - \mathbf{b}\|^2. \quad (2.22)$$

Using the identity $\|\mathbf{v}\|^2 = \mathbf{v}^\top \mathbf{v}$, we can rewrite the objective as

$$E(\mathbf{z}) = (\mathbf{A}z - \mathbf{b})^\top (\mathbf{A}z - \mathbf{b}). \quad (2.23)$$

Expanding this expression gives

$$E(\mathbf{z}) = \mathbf{z}^\top \mathbf{A}^\top \mathbf{A}z - 2\mathbf{b}^\top \mathbf{A}z + \mathbf{b}^\top \mathbf{b}. \quad (2.24)$$

Now differentiate with respect to $\mathbf{z}$ and set the result equal to zero:

$$2\mathbf{A}^\top \mathbf{A} \mathbf{z} - 2\mathbf{A}^\top \mathbf{b} = 0. \quad (2.25)$$

Rearranging gives the normal equations

$$\mathbf{A}^\top \mathbf{A} \hat{\mathbf{z}} = \mathbf{A}^\top \mathbf{b}. \quad (2.26)$$

If $\mathbf{A}^\top \mathbf{A}$ is invertible, we multiply both sides by $(\mathbf{A}^\top \mathbf{A})^{-1}$ to isolate the coefficient vector. The fitted coefficients are therefore

$$\hat{\mathbf{z}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}. \quad (2.27)$$

2.3.2 Evaluating the Pseudoinverse

For this particular 3x3 geometry, one can compute

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 9 & 0 & 0 \\ 0 & 6 & 0 \\ 0 & 0 & 6 \end{bmatrix}, \quad (2.28)$$

so

$$(\mathbf{A}^\top \mathbf{A})^{-1} = \begin{bmatrix} \frac{1}{9} & 0 & 0 \\ 0 & \frac{1}{6} & 0 \\ 0 & 0 & \frac{1}{6} \end{bmatrix}. \quad (2.29)$$

Thus the pseudoinverse is

$$\mathbf{P} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top. \quad (2.30)$$

And we evaluate it as:

$$\mathbf{P} = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} & \frac{1}{9} & \frac{1}{9} & \frac{1}{9} & \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ -\frac{1}{6} & 0 & \frac{1}{6} & -\frac{1}{6} & 0 & \frac{1}{6} & -\frac{1}{6} & 0 & \frac{1}{6} \\ -\frac{1}{6} & -\frac{1}{6} & -\frac{1}{6} & 0 & 0 & 0 & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \end{bmatrix}. \quad (2.31)$$

The second row of $\mathbf{P}$ gives the weights for c_1 , the x -derivative coefficient, and the third row gives the weights for c_2 , the y -derivative coefficient. For example, the weight vector for c_1 is

$$\mathbf{p}_{c_1}^\top = \left[-\frac{1}{6}, 0, \frac{1}{6}, -\frac{1}{6}, 0, \frac{1}{6}, -\frac{1}{6}, 0, \frac{1}{6} \right]. \quad (2.32)$$

And the weight vector for c_2 is

$$\mathbf{p}_{c_2}^\top = \left[-\frac{1}{6}, -\frac{1}{6}, -\frac{1}{6}, 0, 0, 0, \frac{1}{6}, \frac{1}{6}, \frac{1}{6} \right]. \quad (2.33)$$

Then, we can find the fitted derivative coefficient for c_1 is

$$\hat{c}_1 = \mathbf{p}_{c_1}^\top \mathbf{b}. \quad (2.34)$$

and the fitted derivative coefficient for c_2 is

$$\hat{c}_2 = \mathbf{p}_{c_2}^\top \mathbf{b}. \quad (2.35)$$

Since we do not have the exact value for c_1 and c_2 , we can only write the estimates as approximations, hence the usage of the symbols $\hat{c}_1$ and $\hat{c}_2$.

2.4 Extracting the Convolution Filters

2.4.1 Writing the Derivative Estimates as Weighted Sums

Finally, converting the results from the polynomial fit into derivative filters, we write out the explicit expressions:

$$\hat{c}_1 = \left(-\frac{1}{6}\right)I_{-1,-1} + 0I_{0,-1} + \left(\frac{1}{6}\right)I_{1,-1} \quad (2.36)$$

$$+ \left(-\frac{1}{6}\right)I_{-1,0} + 0I_{0,0} + \left(\frac{1}{6}\right)I_{1,0} \quad (2.37)$$

$$+ \left(-\frac{1}{6}\right)I_{-1,1} + 0I_{0,1} + \left(\frac{1}{6}\right)I_{1,1}. \quad (2.38)$$

2.4.2 Rearranging the Weights into 3x3 Stencils

Because the entries of $\mathbf{b}$ were stacked in row-major order, the weights in $\mathbf{p}_{c_1}^\top$ and $\mathbf{p}_{c_2}^\top$ can be placed back into that same 3×3 geometry. In other words, the weighted sum for each derivative estimate can be rewritten as a local stencil whose entries sit at the same pixel locations as the original patch. Symbolically, this rearrangement is

$$\mathbf{K}_x = \text{reshape}_{3,3}(\mathbf{p}_{c_1}^\top), \quad \mathbf{K}_y = \text{reshape}_{3,3}(\mathbf{p}_{c_2}^\top). \quad (2.39)$$

Once that reshaping is done, the least-squares derivative estimate is no longer just a vector expression. It is an ordinary 3×3 filter that can be applied directly to the image.

2.4.3 The Resulting 3x3 Kernels

And converting into standard 3x3 derivative filter format, we have

$$\mathbf{K}_x = \begin{bmatrix} -\frac{1}{6} & 0 & \frac{1}{6} \\ -\frac{1}{6} & 0 & \frac{1}{6} \\ -\frac{1}{6} & 0 & \frac{1}{6} \end{bmatrix}. \quad (2.40)$$

For c_2 , we have the same idea, but rotated by 90 degrees:

$$\hat{c}_2 = \left(-\frac{1}{6}\right)I_{-1,-1} + \left(-\frac{1}{6}\right)I_{0,-1} + \left(-\frac{1}{6}\right)I_{1,-1} \quad (2.41)$$

$$+ 0I_{-1,0} + 0I_{0,0} + 0I_{1,0} \quad (2.42)$$

$$+ \left(\frac{1}{6}\right)I_{-1,1} + \left(\frac{1}{6}\right)I_{0,1} + \left(\frac{1}{6}\right)I_{1,1}. \quad (2.43)$$

And the corresponding 3x3 filter is

$$\mathbf{K}_y = \begin{bmatrix} -\frac{1}{6} & -\frac{1}{6} & -\frac{1}{6} \\ 0 & 0 & 0 \\ \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \end{bmatrix}. \quad (2.44)$$

2.4.4 Combining the Horizontal and Vertical Responses

Once $\mathbf{K}_x$ and $\mathbf{K}_y$ are known, they can be used exactly like any other pair of derivative filters. Applying them to the image gives the horizontal and vertical responses

$$G_x = \mathbf{K}_x * I, \quad G_y = \mathbf{K}_y * I. \quad (2.45)$$

From those two responses, we form the usual gradient magnitude

$$G = \sqrt{G_x^2 + G_y^2}, \quad (2.46)$$

and the corresponding gradient orientation

$$\theta_g = \text{atan2}(G_y, G_x). \quad (2.47)$$

In other words, after the polynomial fit has been collapsed into $\mathbf{K}_x$ and $\mathbf{K}_y$, the rest of the pipeline is just the standard gradient-based edge-detection workflow.

3 Generalization To An $N \times N$ Square

This $N \times N$ formulation is probably best understood as the standard two-dimensional Savitzky–Golay least-squares construction written in a form convenient for later sections [4]. We restate it here so that the derivation remains self-contained. The same polynomial basis, data vector, and pseudoinverse logic from the 3×3 case carry over directly.

Now suppose that instead of a 3×3 patch, we use a general $N \times N$ square patch centered at the pixel of interest. To make sure there really is a center pixel, we assume that N is odd. Then we define the half-width

$$h = \frac{N - 1}{2}. \quad (3.1)$$

With this notation, the local coordinates in the square support are simply

$$(x, y), \quad x, y \in \{-h, -h + 1, \dots, h - 1, h\}. \quad (3.2)$$

This is just the centered $N \times N$ window around the pixel of interest. Since there are N possible x -values and N possible y -values, the total number of pixels in the support is

$$N_p = N^2. \quad (3.3)$$

To write the least-squares system compactly, we now enumerate those same grid points as

$$(x_i, y_i), \quad i = 1, 2, \dots, N_p. \quad (3.4)$$

3.1 The Monomial Basis and Unknown Coefficients

For a polynomial of degree d , we can define the monomial basis vector.

$$\varphi_{d(x,y)} = \left[1, x, y, \frac{x^2}{2}, xy, \frac{y^2}{2}, \dots \right]^\top, \quad (3.5)$$

This vector is just a compact way to list all polynomial terms up to degree d . For example, from Equation 2.2, we see that a degree-1 polynomial has the basis $[1, x, y]^\top$, where we have the coefficients c_0 , c_1 , and c_2 corresponding to these terms. Additionally, from Equation 2.3, the monomial basis for a degree-2 polynomial is $\left[1, x, y, \frac{x^2}{2}, xy, \frac{y^2}{2} \right]^\top$.

In other words, the vector $\varphi_{d(x,y)}$ simply collects every monomial term that can appear in a 2D polynomial of total degree at most d . Each entry of this basis will have one corresponding unknown coefficient in the polynomial fit. So the length of $\varphi_{d(x,y)}$ is exactly the number of coefficients we must solve for. We denote that number by M .

3.1.1 Counting the Basis Terms

To see where this count comes from, group the monomials by total degree. For total degree 0, there is only 1 term, namely 1. For total degree 1, there are 2 terms, namely x and y . For total degree 2, there are 3 terms, namely x^2 , xy , and y^2 . In general, for total degree n , the admissible monomials are $x^n, x^{n-1}y, x^{n-2}y^2, \dots, y^n$, so there are $n + 1$ of them.

Since the basis includes every total degree from 0 up to d , the total number of basis terms is

$$M = \sum_{n=0}^d (n+1) = 1 + 2 + 3 + \dots + (d+1). \quad (3.6)$$

Using the standard formula for the sum of the first $d+1$ positive integers, this becomes

$$M = (d+1) \frac{d+2}{2}. \quad (3.7)$$

3.1.2 Minimum Sample Requirement

Now that we have acquire M through such painful means, we can now define the minimum data requirement for the least-squares fit. Since we have M unknown coefficients, we need at least M equations to solve for them. If we recall from Section 2.1.2, each pixel in the support gives us one equation, so we need at least M pixels in the support. In more personable terms, the number of pixels in the support must be greater than (or equal to) the number of coefficients.

$$N_p \geq M. \quad (3.8)$$

3.2 Building the General Linear System

3.2.1 The Coefficient Vector and Design Matrix

For the generalized $N \times N$ case, the unknown coefficient vector is

$$\mathbf{z} = [c_0, c_1, c_2, \dots, c_{M-1}]^\top, \quad (3.9)$$

where these coefficients multiply the monomials collected in $\varphi_{d(x,y)}$.

Anyway, getting back on track, we can now write the design matrix $\mathbf{A}$ for the generalized $N \times N$ case. Each row corresponds to one sampled pixel in the support, and each column corresponds to one monomial term in the basis. The i -th row is simply the basis vector evaluated at the coordinates of the i -th pixel.

$$\mathbf{A} = \begin{bmatrix} \varphi_{d(x_1, y_1)}^\top \\ \varphi_{d(x_2, y_2)}^\top \\ \vdots \\ \varphi_{d(x_{N_p}, y_{N_p})}^\top \end{bmatrix}. \quad (3.10)$$

3.2.2 The Data Vector and System Form

To complete the linear system, we stack the observed pixel intensities from those same sample locations, in the same order used for the rows of $\mathbf{A}$.

$$\mathbf{b} = [f(x_1, y_1), f(x_2, y_2), \dots, f(x_{N_p}, y_{N_p})]^\top. \quad (3.11)$$

Using the same ordering in both objects, row i of $\mathbf{A}$ is built from the coordinates (x_i, y_i) , and entry i of $\mathbf{b}$ is the measured intensity at that same location. With those definitions, the generalized polynomial-fitting problem is

$$\mathbf{A}\mathbf{z} \approx \mathbf{b}. \quad (3.12)$$

3.3 Solving for the Derivative Weights

3.3.1 The Pseudoinverse Map

This system is traditionally overdetermined by design (and we will later discuss the reason for such and a proof will be provided), so we solve for the coefficient vector in the least-squares sense. The fitted coefficient vector is

$$\hat{\mathbf{z}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b} = \mathbf{P} \mathbf{b}. \quad (3.13)$$

Where $\hat{\mathbf{z}}$ means the estimated coefficient vector. We also define

$$\mathbf{P} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \quad (3.14)$$

3.3.2 Extracting the First-Derivative Rows

The matrix $\mathbf{P}$ is the pseudoinverse that maps sampled intensities directly to fitted coefficients. If the basis is ordered as $[1, x, y, \dots]^\top$, then the coefficient of x is the estimate of the first derivative in the x -direction, and the coefficient of y is the estimate of the first derivative in the y -direction. Those coefficients are entries of $\hat{\mathbf{z}}$, so the corresponding rows of $\mathbf{P}$ are the two derivative weight vectors

$$\mathbf{p}_{f_x}^\top, \quad (3.15)$$

and

$$\mathbf{p}_{f_y}^\top. \quad (3.16)$$

Applying those rows to the data vector gives the derivative estimates

$$\hat{f}_x = \mathbf{p}_{f_x}^\top \mathbf{b}. \quad (3.17)$$

$$\hat{f}_y = \mathbf{p}_{f_y}^\top \mathbf{b}. \quad (3.18)$$

3.3.3 Arranging the Rows into Square Stencils

Because the support is a square and the samples can be ordered in row-major form, the derivative rows can be reshaped directly back into $\mathbf{N} \times \mathbf{N}$ filter kernels:

$$\mathbf{K}_x^{\text{square}} = \text{reshape}_{N,N}(\mathbf{p}_{f_x}^\top), \quad \mathbf{K}_y^{\text{square}} = \text{reshape}_{N,N}(\mathbf{p}_{f_y}^\top). \quad (3.19)$$

This is the exact analogue of the 3×3 construction in Section 2.4.2, except now the support size is arbitrary. The polynomial fit still produces two ordinary stencils, one for the horizontal derivative and one for the vertical derivative.

3.3.4 Combining the Square-Support Responses

Once those square-support kernels are formed, they are applied exactly as in the small example. The two filter outputs are

$$G_x^{\text{square}} = \mathbf{K}_x^{\text{square}} * I, \quad G_y^{\text{square}} = \mathbf{K}_y^{\text{square}} * I. \quad (3.20)$$

The corresponding edge-strength measure is

$$G^{\text{square}} = \sqrt{(G_x^{\text{square}})^2 + (G_y^{\text{square}})^2}, \quad (3.21)$$

and the associated orientation estimate is

$$\theta^{\text{square}} = \text{atan2}(G_y^{\text{square}}, G_x^{\text{square}}). \quad (3.22)$$

So the logic is exactly the same as in the 3×3 example. First choose sample locations. Then evaluate the monomial basis there to build $\mathbf{A}$. Then stack the measured intensities into $\mathbf{b}$ in the same order. Solving the least-squares problem gives $\hat{\mathbf{z}}$, and the derivative rows of the pseudoinverse become square filter stencils whose responses can then be combined into a gradient magnitude and orientation.

4 Visualizing the Square-Support Filters

For a visual example, we can inspect the actual square-support kernels produced by this construction. If we choose $N = 15$, then the corresponding half-width is

$$h = \frac{N - 1}{2} = 7. \quad (4.1)$$

With that support fixed, Figure 1, Figure 2, and Figure 3 show the square-support derivative stencils for polynomial degrees $d = 1$, $d = 3$, and $d = 5$.

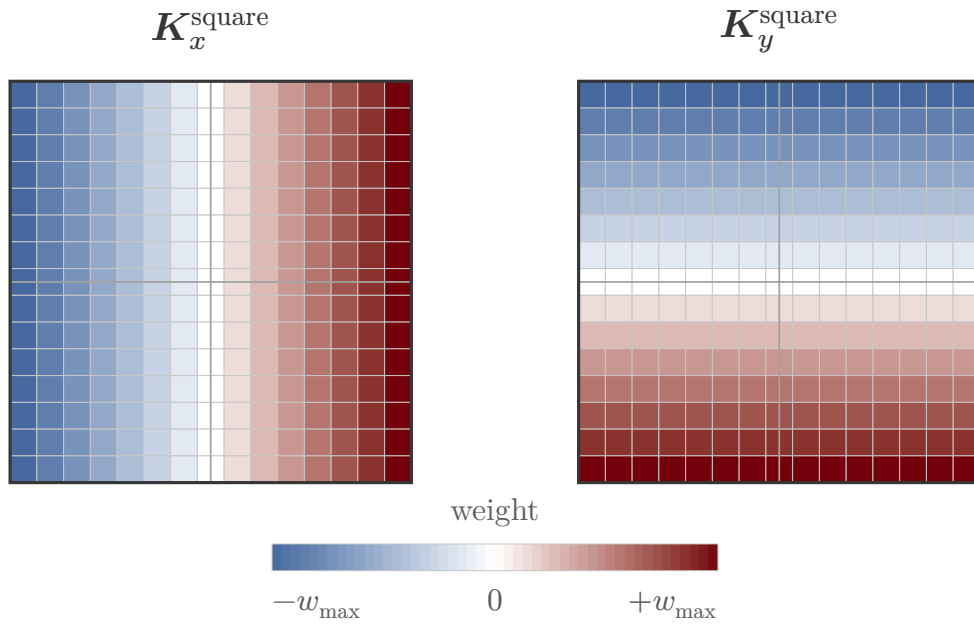


Figure 1: Square-support derivative kernels for a 15×15 neighborhood with polynomial degree $d = 1$. The left panel shows $\mathbf{K}_x^{\text{square}}$ and the right panel shows $\mathbf{K}_y^{\text{square}}$.

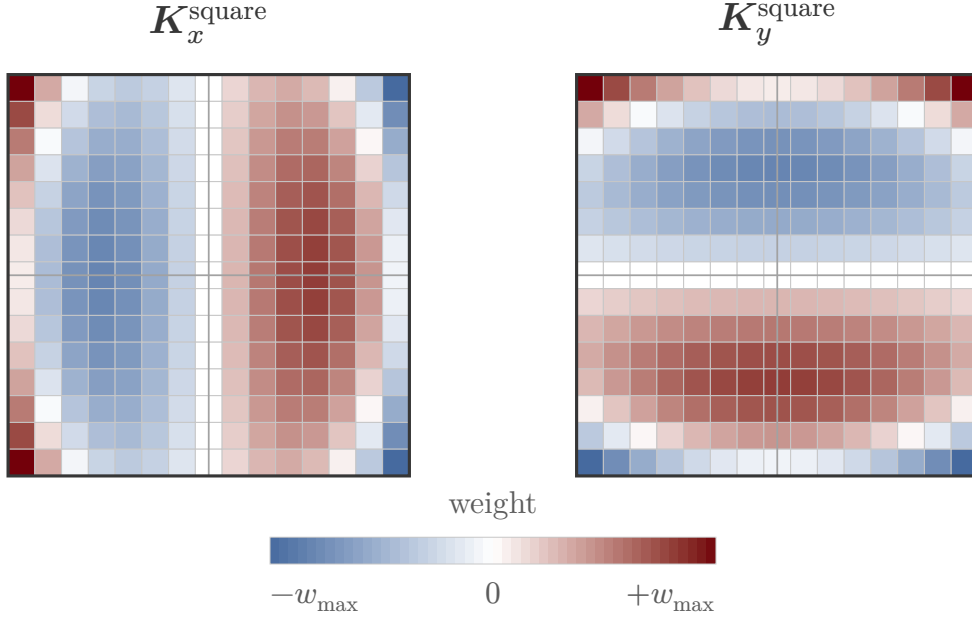


Figure 2: Square-support derivative kernels for a 15×15 neighborhood with polynomial degree $d = 3$.

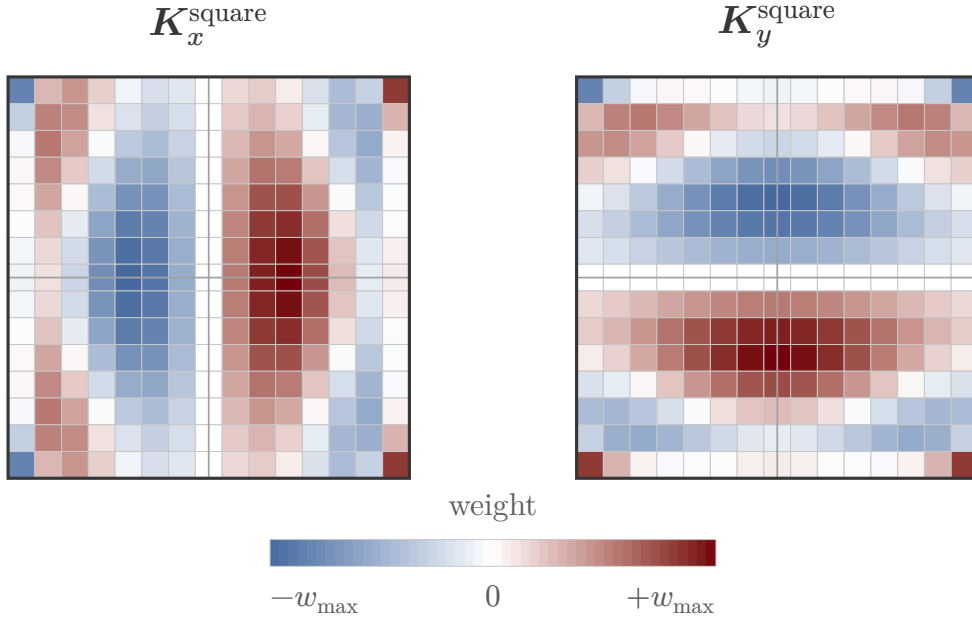


Figure 3: Square-support derivative kernels for a 15×15 neighborhood with polynomial degree $d = 5$.

Several points are worth noting. First, the degree-1 kernels have the simplest possible structure. They form a monotone left-to-right ramp for K_x^{square} and the corresponding top-to-bottom ramp for K_y^{square} . Second, once the degree is increased to $d = 3$, the kernels become more concentrated near the middle of the support and develop small opposite-sign boundary lobes near the outer edges. That same trend becomes even more pronounced for $d = 5$, where the central positive and negative bands strengthen further and the outer oscillatory structure is easier to see. Finally, each panel retains the expected square-support symmetry. In every case, the y -derivative kernel is the transpose of the x -derivative kernel, so the two heatmaps contain the same structure rotated by ninety degrees.

There is also a useful parity observation hiding in these figures. For a centered symmetric square support, the first-derivative stencil changes only when the polynomial basis gains new terms with

the symmetry needed to contribute to f_x or f_y . As a result, the square-support derivative kernels tend to appear in odd-even pairs. In practice, this means that $d = 1$ and $d = 2$ look the same, $d = 3$ and $d = 4$ look the same, and $d = 5$ and $d = 6$ look the same. This is precisely why the visual comparison above uses $d = 1$, $d = 3$, and $d = 5$ rather than the intervening even degrees. Those odd degrees are the ones at which the square-support first-derivative stencils visibly change.

This figure also helps explain why the support geometry matters so much. Even before the circular and rotated constructions are introduced, the square support already imposes a visible spatial pattern on where the derivative estimate places its positive and negative weight. The next section changes only the support geometry, allowing us to isolate how much of the filter shape comes from the neighborhood itself rather than from the least-squares logic.

5 Adapting to a Circular Support Region

5.1 Restricting the Sample Locations

Classical Savitzky–Golay filters and most of their descendants are usually presented on rectangular, axis-aligned neighborhoods, largely because those supports make both the algebra and the implementation straightforward [1], [4]. The circular case uses the same least-squares construction, but changes the set of sample locations so that the neighborhood is defined by radius rather than by axis-aligned extent.

We can derive this circular version by restricting the samples to those that satisfy the standard circle constraint:

$$x_i^2 + y_i^2 \leq r^2. \quad (5.1)$$

5.2 Reusing the Least-Squares Construction

Now, the simplification from the prior section of redefining the N_p rather than N^2 will save us some time. Defining the data vector to only contain the pixel values inside that circular support, we derive $\mathbf{b}$ as

$$\mathbf{b} = [I_1, I_2, \dots, I_{N_p}]^\top, \quad (5.2)$$

and the design matrix keeps only the corresponding rows

$$\mathbf{A} = \begin{bmatrix} \varphi_{d(x_1, y_1)}^\top \\ \varphi_{d(x_2, y_2)}^\top \\ \vdots \\ \varphi_{d(x_{N_p}, y_{N_p})}^\top \end{bmatrix}, \quad (5.3)$$

where the coordinates now come from the circular neighborhood rather than the full square. Naturally, the least-squares solution remains unchanged

$$\hat{\mathbf{z}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}. \quad (5.4)$$

and again, the derivative estimates are obtained from rows of the pseudoinverse

$$\hat{f}_x = \mathbf{p}_{f_x}^\top \mathbf{b}. \quad (5.5)$$

$$\hat{f}_y = \mathbf{p}_{f_y}^\top \mathbf{b}. \quad (5.6)$$

5.2.1 Embedding the Circular Weights into Stencils

The vectors $\mathbf{p}_{f_x}^\top$ and $\mathbf{p}_{f_y}^\top$ still define discrete filters, but now only at the sample locations that lie inside the circle. To implement them as image stencils, we place those weights at the corresponding

coordinates of the bounding square and assign zero weight to locations outside the circular support. That produces two circular-support masks, which we denote by $\mathbf{K}_x^{\text{circ}}$ and $\mathbf{K}_y^{\text{circ}}$.

5.2.2 Combining the Circular-Support Responses

Once those circular-support stencils are formed, the corresponding filter outputs are

$$G_x^{\text{circ}} = \mathbf{K}_x^{\text{circ}} * I, \quad G_y^{\text{circ}} = \mathbf{K}_y^{\text{circ}} * I. \quad (5.7)$$

From them we define the circular-support gradient magnitude

$$G^{\text{circ}} = \sqrt{(G_x^{\text{circ}})^2 + (G_y^{\text{circ}})^2}, \quad (5.8)$$

and the associated orientation estimate

$$\theta^{\text{circ}} = \text{atan2}(G_y^{\text{circ}}, G_x^{\text{circ}}). \quad (5.9)$$

So the circle support is just a different choice of sample locations, but the underlying least-squares logic is exactly the same. The resulting filter weights are different because the sample locations are different, but the mathematical construction is identical. We will show a visual representation of the difference of these two supports later, but for now, the main takeaway is that the circular support is just a different set of sample locations, and the least-squares logic applies in exactly the same way.

6 Visualizing the Circular-Support Filters

To make that geometric change concrete, it is useful to visualize the corresponding circular-support derivative stencils. For a fair comparison with the square case, we keep the same bounding window size $N = 15$ and therefore the same half-width $h = 7$. The circular support is then defined as the exact integer lattice disk

$$x_i^2 + y_i^2 \leq 7^2. \quad (6.1)$$

With this choice, the support contains

$$N_p = 149 \quad (6.2)$$

sample locations. That count is not chosen arbitrarily. It is simply the number of integer grid points inside the radius-7 disk. This exact construction is important because it preserves isotropy in the discrete support itself. By contrast, if one selects an arbitrary odd N_p by taking the nearest pixels in distance order, the final few accepted points can depend on tie-breaking among equally distant lattice sites, which can introduce a slight directional bias into what is supposed to be a circular neighborhood.

With that isotropic support fixed, Figures Figure 4, Figure 5, and Figure 6 show the circular-support derivative stencils for polynomial degrees $d = 1$, $d = 3$, and $d = 5$.

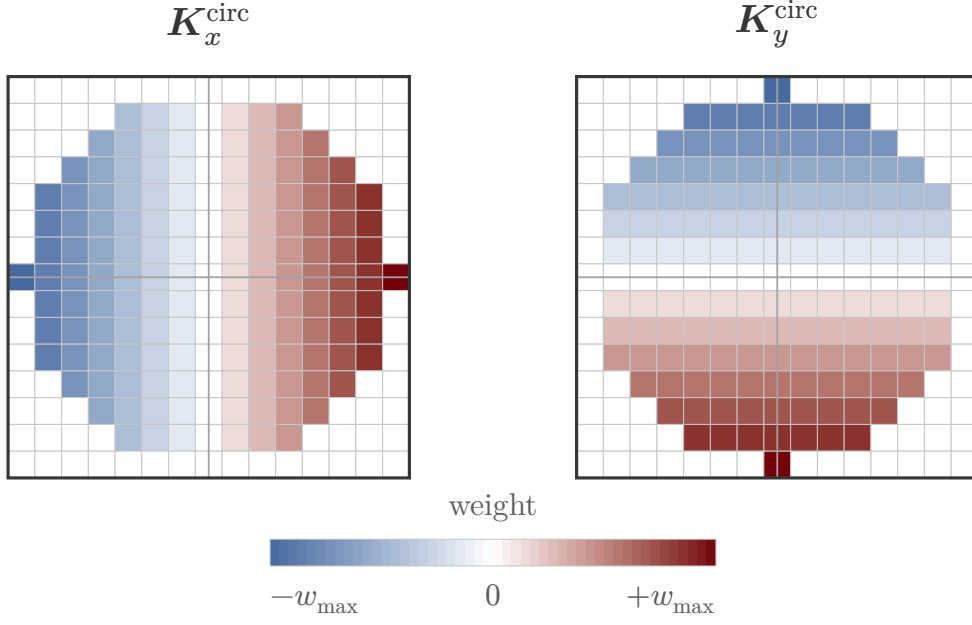


Figure 4: Circular-support derivative kernels for the exact radius-7 disk inside a 15 x 15 bounding box, with polynomial degree $d = 1$. The left panel shows K_x^{circ} and the right panel shows K_y^{circ} .

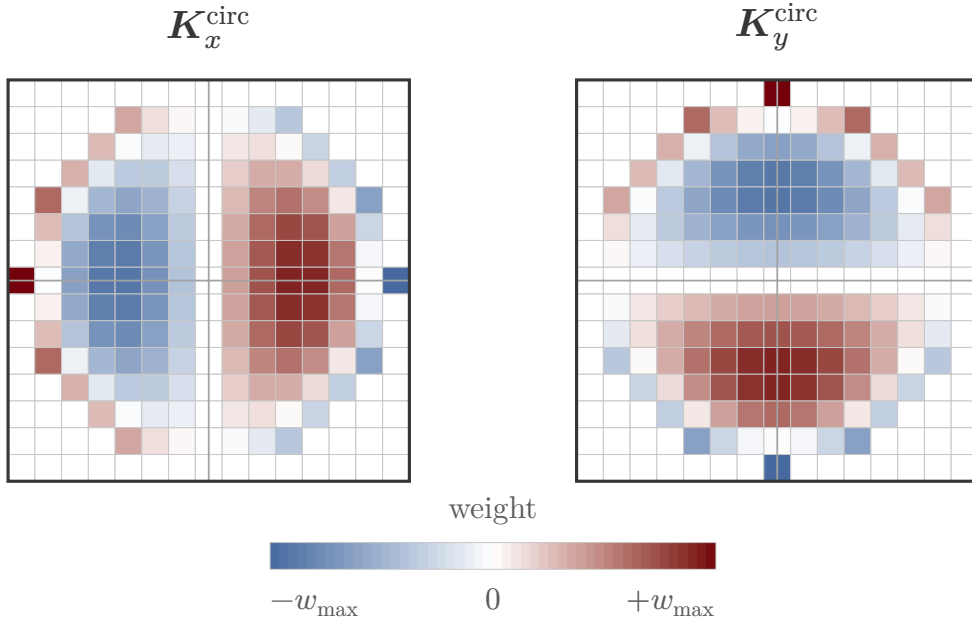


Figure 5: Circular-support derivative kernels for the exact radius-7 disk with polynomial degree $d = 3$.

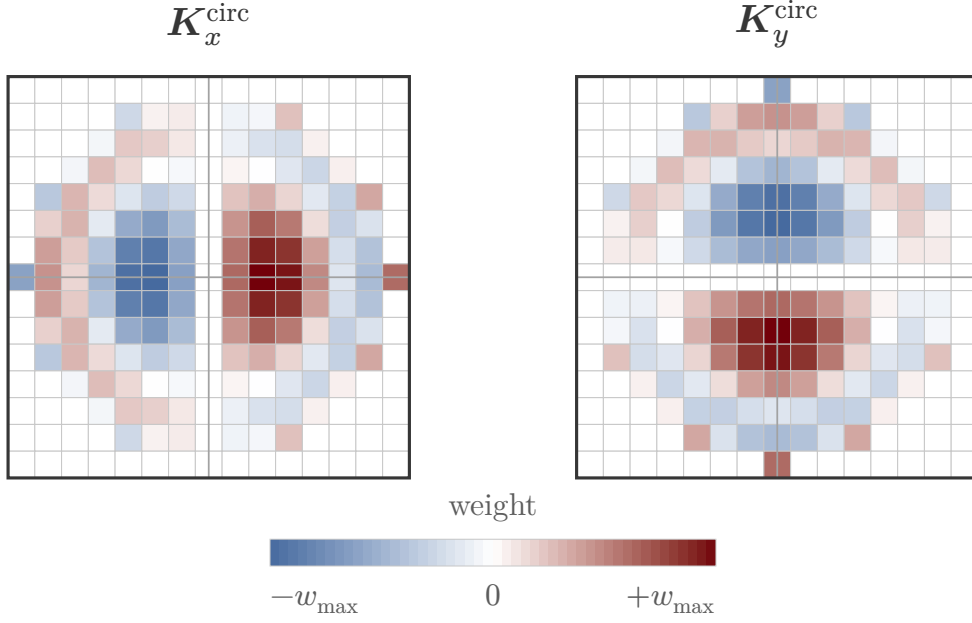


Figure 6: Circular-support derivative kernels for the exact radius-7 disk with polynomial degree $d = 5$.

The same odd-even pairing seen in the square-support case appears here as well. For this centered isotropic support, the first-derivative stencils again change only when the polynomial basis gains new terms with the correct symmetry to contribute to f_x or f_y . Consequently, the visually distinct cases occur at $d = 1$, $d = 3$, and $d = 5$, while the intervening even degrees produce the same first-derivative stencils as their preceding odd neighbors. This is why the circular visualizations use the same degree sequence as the square ones.

The circular figures also make the geometric difference from the square support easy to see. The kernels are no longer forced to occupy the corners of the 15×15 bounding box, so the weight distribution conforms more closely to a radially symmetric neighborhood. The x - and y -derivative stencils still remain transposes of one another, but the support itself is now isotropic rather than axis-aligned. This gives a cleaner baseline for the next section, where the coordinate system itself is rotated and only the derivative in the candidate normal direction is retained.

7 Rotating the Circular Support

The next step toward the WVF is not to change the support again, but to rotate the derivative direction within that same isotropic circular neighborhood. To illustrate this, we take the same support from Figure 5, but rotate it across some set of angles, as seen in Figure 7 where we fix the exact radius-7 disk inside the 15×15 bounding box and shows the directional stencil obtained when the local coordinate system is rotated to $\theta = 0^\circ$, 30° , 60° , and 90° . For this illustration, we use the representative odd degree $d = 3$.

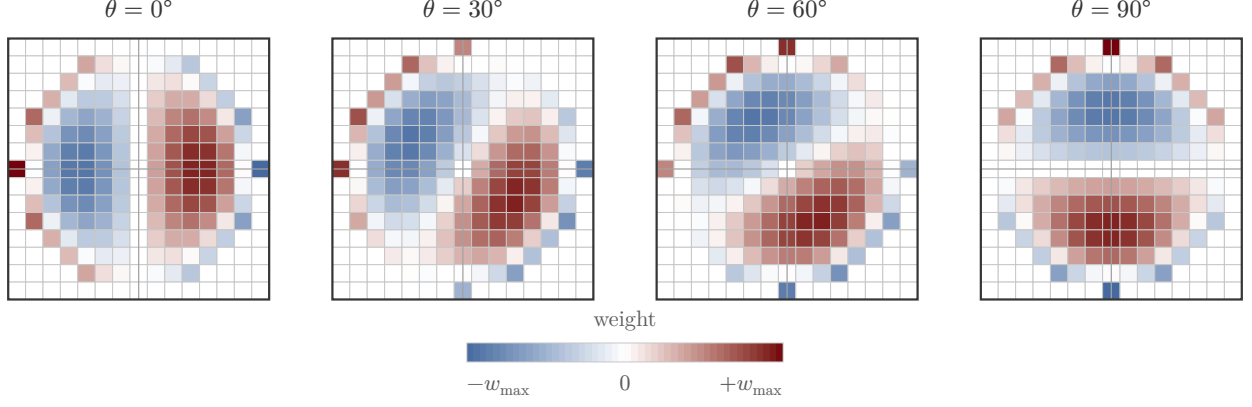


Figure 7: Orientation sweep for the exact circular-support stencil with $N = 15$, $h = 7$, and polynomial degree $d = 3$. Each panel shows the directional derivative stencil obtained from the same isotropic disk after rotating the local coordinate system to a new angle θ .

The underlying support stays circular, but the positive and negative lobes of the derivative stencil rotate with the chosen coordinate system. In this case, the neighborhood itself remains isotropic while the extracted derivative direction changes. That is precisely the idea used by the WVF. Rather than relying only on the fixed horizontal and vertical derivatives, the filter is evaluated in several candidate directions and the directional derivative *most* aligned with the candidate normal is retained.

8 The Wide View Filter Formulation

The WVF keeps the same least-squares construction introduced in Section 5, but changes the coordinate system in which the derivative is taken. Once that rotated directional stencil has been defined for one candidate angle, the filter is evaluated over a finite set of candidate angles and the strongest directional response is retained.

8.1 Rotated Local Coordinates and the Directional Stencil

Unlike the Savitzky–Golay derivative filters derived in the previous sections, the WVF does not use a paired horizontal and vertical response. In the square and circular constructions, Equation 3.17 and Equation 3.18 showed that the fit naturally produces both $\hat{f}_x$ and $\hat{f}_y$. Here the objective is different. The WVF rotates the local coordinates first, and then keeps only the derivative aligned with the rotated x' direction. The orthogonal y' coefficient may still appear in the fitted coefficient vector, but it is not used when the WVF forms its final response.

8.1.1 The Rotation Step

The first change is to rotate the local coordinates so that the derivative is taken in a candidate normal direction:

$$x_{i'} = \Delta x_i \cos \theta + \Delta y_i \sin \theta, \quad (8.1)$$

$$y_{i'} = -\Delta x_i \sin \theta + \Delta y_i \cos \theta. \quad (8.2)$$

8.1.2 The Orientation-Dependent Stencil

The second change is that the WVF uses the circular support from Section 5 rather than the square support from Section 3. Once the rotated coordinates in Equation 8.1 and Equation 8.2 are fixed, the same least-squares logic used in Equation 5.4 still applies. Using Equation 8.1 and Equation 8.2, the i -th row of the design matrix becomes $\varphi_{d(x_{i'}, y_{i'})}^\top$ instead of $\varphi_{d(x_i, y_i)}^\top$. The data vector $\mathbf{b}$ is unchanged, because it still contains the same sampled pixel intensities as in Equation 5.4. With those definitions, the fitted coefficient vector is

$$\hat{\mathbf{z}} = \mathbf{P}_\theta \mathbf{b}, \quad (8.3)$$

Using Equation 8.3, the row of $\mathbf{P}_\theta$ corresponding to the x' coefficient gives the directional derivative estimate

$$\hat{f}_{x'} = \mathbf{p}_{f_{x'}}^\top \mathbf{b}. \quad (8.4)$$

Equation 8.4 shows that the directional derivative is again a weighted sum of the sampled intensities, just as in Equation 3.17 and Equation 5.5. Therefore the coefficients in $\mathbf{p}_{f_{x'}}^\top$ are the WVF stencil for the candidate angle θ . Each candidate angle has its own stencil, and applying that stencil gives the WVF estimate of the derivative in the rotated x' direction.

8.2 Discrete Orientation Sampling

The WVF does not stop after constructing one stencil. Instead, it constructs one orientation-dependent stencil for each sampled angle. Since the rotated x' axis at angle $\theta + \pi$ points in the opposite direction from the rotated x' axis at angle θ , the directional derivative changes sign but not magnitude. Because of that sign symmetry, it is sufficient to sample candidate angles on the interval $[0, \pi)$. We therefore define the sampled angle set by

$$\Theta_{N_s} = \left\{ \theta_k = k \frac{\pi}{N_s} : k = 0, 1, \dots, N_s - 1 \right\}. \quad (8.5)$$

Using Equation 8.4, the directional response at the candidate angle θ_k is

$$R(\theta_k) = \mathbf{p}_{f_{x'}}(\theta_k)^\top \mathbf{b}. \quad (8.6)$$

Equation 8.6 makes the WVF comparison step explicit. For every sampled angle in Equation 8.5, the filter evaluates one weighted sum of the sampled intensities. Because the WVF is built from the x' row of $\mathbf{P}_\theta$ in Equation 8.4, only the rotated x' derivative is used when the candidate angles are compared.

8.3 Selection of the Winning Direction

Once the candidate responses in Equation 8.6 have been computed, the WVF keeps the angle with the largest absolute directional response. In symbols,

$$\theta_* = \arg \max_{\theta_k \in \Theta_{N_s}} |R(\theta_k)|. \quad (8.7)$$

Using Equation 8.7, the WVF magnitude returned at that pixel is

$$G^{\text{WVF}} = |R(\theta_*)|. \quad (8.8)$$

Equation 8.5, Equation 8.6, Equation 8.7, and Equation 8.8 together complete the WVF definition. The filter first samples a discrete set of candidate angles, then evaluates the directional derivative in the rotated x' direction for each angle, and finally returns the angle and magnitude associated with the strongest absolute response.

9 Low-Degree Overdetermined Fits

The WVF construction above is still built from a local polynomial fit, so the quality of the filter depends on the quality of that fit. For that reason, the next step is to explain why low-degree fits on well-sampled supports are preferable when the goal is stable first-derivative estimation.

9.1 Exact Recovery for Affine Data

The cleanest case is an affine intensity field, which simply means the intensity is a linear function of the spatial coordinates. We can define the intensity function of such a field as

$$f_{\text{aff}(x,y)} = a_0 + a_1 x + a_2 y. \quad (9.1)$$

If the sampled data are generated exactly from Equation 9.1, then the sampled intensity vector can be written as

$$\mathbf{b}_0 = \mathbf{A}_1 \mathbf{z}_{\text{aff}}, \quad (9.2)$$

where $\mathbf{A}_1$ is the degree-1 design matrix and $\mathbf{z}_{\text{aff}} = [a_0, a_1, a_2]^\top$. Using the least-squares map from Equation 3.13 and the pseudoinverse definition from Equation 3.14, the fitted coefficient vector is

$$\hat{\mathbf{z}} = \mathbf{P}_1 \mathbf{b}_0 = (\mathbf{A}_1^\top \mathbf{A}_1)^{-1} \mathbf{A}_1^\top \mathbf{A}_1 \mathbf{z}_{\text{aff}} = \mathbf{z}_{\text{aff}}. \quad (9.3)$$

Equation 9.3 shows that the degree-1 fit recovers the affine coefficients exactly whenever the data are noiseless and truly affine. Since the coefficient of x in Equation 9.1 is a_1 and the coefficient of y is a_2 , Equation 9.3 implies

$$\hat{f}_x = a_1, \quad \hat{f}_y = a_2. \quad (9.4)$$

Equation 9.4 is the exact-recovery statement we need. For affine data, the degree-1 polynomial already contains the entire local signal, so higher polynomial degree does not add any new first-derivative information in the noiseless case.

9.2 The Affine Model as a Local Approximation

The affine model in Equation 9.1 is not intended as a global model of the image, rather it is a local approximation. We assume local smoothness because the goal is to estimate local derivatives and derivatives and Taylor expansions are only meaningful where the intensity field is sufficiently regular. Under that assumption, the first-order Taylor expansion provides the affine approximation we use.

If the underlying intensity field is smooth near the center pixel, then its first-order Taylor expansion about that center point is

$$f(x, y) = f(0, 0) + f_{x(0,0)}x + f_{y(0,0)}y + r_2(x, y), \quad (9.5)$$

where $r_2(x, y)$ collects every quadratic and higher-order term. Using Taylor's theorem, the magnitude of that remainder satisfies a bound of the form

$$|r_2(x, y)| \leq \left(\frac{1}{2}\right) \sup_{(u,v) \in \Omega_h} \|\mathbf{H}_{f(u,v)}\| \|(x, y)\|^2, \quad (9.6)$$

where Ω_h is the local support window and $\mathbf{H}_f$ is the Hessian of the intensity field. Equation 9.5 shows that the affine term is the leading local approximation, while Equation 9.6 shows that the approximation error grows quadratically with distance from the expansion point. Over a small enough support, the tangent line is the first-order approximation to the smooth profile, and Equation 9.6 tells us how the neglected curvature enters.

9.3 Noise Propagation Through the Derivative Stencil

To discuss robustness, we now write the sampled data as the sum of a clean signal and additive noise, where $\mathbf{b}_0$ is the clean signal and $\boldsymbol{\varepsilon}$ is the noise vector

$$\mathbf{b} = \mathbf{b}_0 + \boldsymbol{\varepsilon}, \quad (9.7)$$

with E denoting the expectation operator. We assume that the noise is zero-mean and isotropic, so its mean and covariance are

$$E[\boldsymbol{\varepsilon}] = 0, \quad \text{Cov}(\boldsymbol{\varepsilon}) = \sigma^2 \mathbf{I}. \quad (9.8)$$

Using the derivative-row relation in Equation 3.17, the noisy derivative estimate is

$$\hat{f}_x = \mathbf{p}_{f_x}^\top \mathbf{b} = \mathbf{p}_{f_x}^\top \mathbf{b}_0 + \mathbf{p}_{f_x}^\top \boldsymbol{\varepsilon}. \quad (9.9)$$

Taking expectations in Equation 9.9 and using Equation 9.8 gives

$$E[\hat{f}_x] = \mathbf{p}_{f_x}^\top \mathbf{b}_0. \quad (9.10)$$

Equation 9.10 shows that zero-mean noise does not change the average fitted derivative. The remaining effect of noise is therefore the random spread of the estimator around that average, which is measured by the variance. Using Equation 9.9 and Equation 9.8,

$$\text{Var}(\hat{f}_x) = \mathbf{p}_{f_x}^\top \text{Cov}(\boldsymbol{\varepsilon}) \mathbf{p}_{f_x} = \sigma^2 \mathbf{p}_{f_x}^\top \mathbf{p}_{f_x} = \sigma^2 \|\mathbf{p}_{f_x}\|^2. \quad (9.11)$$

Equation 9.11 gives a direct robustness criterion. If the image noise level σ^2 is fixed, then the only factor controlling the variance of the derivative estimate is $\|\mathbf{p}_{f_x}\|^2$. In this sense, the squared norm of the derivative row is the noise gain of the stencil. Larger derivative-row norms therefore mean stronger noise amplification and a less stable estimate.

9.4 Higher-Degree Fits as Less Overdetermined Systems

The basis-count result in Equation 3.7 shows that the number of unknown coefficients grows with degree, while the minimum-data requirement in Equation 3.8 shows that the support size must remain at least as large as that coefficient count. If the support size N_p is fixed, then increasing the degree must decrease the excess number of samples,

$$E_d = N_p - M_d, \quad (9.12)$$

where M_d is the basis size from Equation 3.7. Equation 9.12 shows that higher degree makes the fit less overdetermined. That loss of redundancy can be written directly in terms of the derivative stencil.

For a fixed support and a fixed degree d , let $\mathbf{A}_d$ be the design matrix and let $\mathbf{e}_{f_x}^d$ be the unit vector that selects the x coefficient. If a degree- d polynomial has coefficient vector $\mathbf{z}_d$, then its sampled values are $\mathbf{A}_d \mathbf{z}_d$. If a stencil $\boldsymbol{\alpha}$ is to recover the x coefficient from those samples for every possible $\mathbf{z}_d$, then we must have

$$\boldsymbol{\alpha}^\top \mathbf{A}_d \mathbf{z}_d = \mathbf{e}_{f_x}^{d\top} \mathbf{z}_d \quad (9.13)$$

for every coefficient vector $\mathbf{z}_d$. Since that identity must hold for every $\mathbf{z}_d$, it is equivalent to the linear constraint

$$\mathbf{A}_d^\top \boldsymbol{\alpha} = \mathbf{e}_{f_x}^d. \quad (9.14)$$

Equation 9.14 states that the stencil reproduces the x coefficient exactly for every polynomial in the degree- d basis. Using the pseudoinverse definition in Equation 3.14, the derivative row is the Moore-Penrose solution of those linear constraints, so among all vectors satisfying Equation 9.14 it is the minimum-norm choice:

$$\mathbf{p}_{f_x}^d = \arg \min_{\boldsymbol{\alpha}} \|\boldsymbol{\alpha}\|^2 \quad \text{subject to} \quad \mathbf{A}_d^\top \boldsymbol{\alpha} = \mathbf{e}_{f_x}^d. \quad (9.15)$$

Now compare degree d and degree $d+1$ on the same support. Since every degree- d polynomial is also a degree- $(d+1)$ polynomial, the degree- $(d+1)$ exactness conditions include all of the degree- d conditions and then add more. Therefore the feasible set defined by Equation 9.14 for degree $d+1$ is a subset of the feasible set for degree d . Using Equation 9.15, the minimum achievable stencil norm cannot decrease when the degree is raised:

$$\|\mathbf{p}_{f_x}^{d+1}\| \geq \|\mathbf{p}_{f_x}^d\|. \quad (9.16)$$

Using Equation 9.11 and Equation 9.16, the derivative variance under the additive noise model cannot decrease when the degree is raised on the same support:

$$\text{Var}(\hat{f}_x^{d+1}) \geq \text{Var}(\hat{f}_x^d). \quad (9.17)$$

Equation 9.17 is the central tradeoff. Higher degree enforces exactness for a larger polynomial space, but it does so by imposing more cancellation constraints on the derivative stencil. Those extra cancellations are what make the higher-degree kernels in Figure 2, Figure 3, Figure 5, and Figure 6 look more oscillatory than their lower-degree counterparts. The higher-degree fit is therefore less overdetermined, more constrained, **and** more noise sensitive, even before one discusses any empirical performance differences.

10 Symmetry, Redundancy, and the Practical Limits of WVF

The WVF is still built from the same local polynomial fit, so the next question is whether rotating the coordinate system creates any new first-order information. Thankfully, the answer can be determined by following the geometry of the support, the chain rule for directional derivatives, and the least-squares objective itself.

10.1 Fixed Circular Support Under Rotation

Let the circular support be the fixed set of sample locations

$$S_r = \{(\Delta x_i, \Delta y_i) \in \mathbb{Z}^2 : \Delta x_i^2 + \Delta y_i^2 \leq r^2\}. \quad (10.1)$$

Using Equation 10.1, the support is chosen before any angle is considered. Using Equation 8.1 and Equation 8.2, the WVF then rotates only the coordinate description of those same sample locations. This means that the sampled intensity vector $\mathbf{b}$ does not change with angle. Only the coordinates used to build the design matrix change.

10.2 Directional Derivatives from the Fitted Gradient

To express the rotated derivative in terms of the fitted gradient, we first invert the rotation in Equation 8.1 and Equation 8.2. The inverse relations are

$$x = x' \cos \theta - y' \sin \theta, \quad (10.2)$$

$$y = x' \sin \theta + y' \cos \theta. \quad (10.3)$$

Using Equation 10.2 and Equation 10.3, the chain rule gives

$$\frac{\partial}{\partial} x' = \cos \theta \frac{\partial}{\partial} x + \sin \theta \frac{\partial}{\partial} y. \quad (10.4)$$

Applying Equation 10.4 to the fitted polynomial at the center point and using the derivative definitions from Equation 3.17 and Equation 3.18 yields

$$\hat{f}_{x'}(\theta) = \cos \theta \hat{f}_x + \sin \theta \hat{f}_y. \quad (10.5)$$

Equation 10.5 shows that once $\hat{f}_x$ and $\hat{f}_y$ are known, the directional derivative in any rotated direction is already determined.

10.3 The Continuous Optimal Direction

Equation 10.5 can be rewritten in amplitude-phase form. Define

$$G = \sqrt{\hat{f}_x^2 + \hat{f}_y^2}, \quad \theta_g = \text{atan2}(\hat{f}_y, \hat{f}_x). \quad (10.6)$$

Using Equation 10.6, the directional derivative in Equation 10.5 becomes

$$\hat{f}_{x'}(\theta) = G \cos(\theta - \theta_g). \quad (10.7)$$

Since the largest possible magnitude of $\cos(\theta - \theta_g)$ is 1, Equation 10.7 implies

$$\max_{\theta \in [0, \pi)} |\hat{f}_{x'}(\theta)| = G = \sqrt{\hat{f}_x^2 + \hat{f}_y^2}. \quad (10.8)$$

Equation 10.8 shows that a continuous search over direction returns exactly the gradient magnitude, while the maximizing direction is the gradient direction θ_g from Equation 10.6. So, in the case where we rotate the WVF in continuous setting, the directional sweep does not add new first-order information. It only re-expresses the fitted gradient already derived from the standard horizontal and vertical fits.

10.4 Rotational Equivariance of the Exact Circular Fit

The previous subsection showed that directional derivatives of one fitted polynomial are already determined by $\hat{f}_x$ and $\hat{f}_y$. We now show that the rotated WVF fit is still that same fitted polynomial, only written in a rotated basis.

Let Π_d denote the space of all bivariate polynomials of total degree at most d . For any polynomial $q \in \Pi_d$, define the rotation map

$$(T_\theta q)(x', y') = q(x' \cos \theta - y' \sin \theta, x' \sin \theta + y' \cos \theta). \quad (10.9)$$

Using Equation 10.9, every monomial in q is evaluated after a linear substitution in x' and y' . A linear substitution does not increase total degree, so $T_\theta q$ is still in Π_d . Therefore Equation 10.9 defines a one-to-one correspondence from Π_d onto itself.

With that notation, the least-squares fit in the original coordinates is

$$q^* = \arg \min_{q \in \Pi_d} \sum_i (q(\Delta x_i, \Delta y_i) - I_i)^2. \quad (10.10)$$

Using the rotated coordinates from Equation 8.1 and Equation 8.2, the WVF fit at angle θ is

$$q_\theta^* = \arg \min_{\tilde{q} \in \Pi_d} \sum_i (\tilde{q}(x_{i'}, y_{i'}) - I_i)^2. \quad (10.11)$$

Now take any candidate polynomial $q \in \Pi_d$ in Equation 10.10 and map it to $\tilde{q} = T_\theta q$. Using Equation 10.9 together with Equation 8.1 and Equation 8.2, the residual of $\tilde{q}$ at the rotated sample $(x_{i'}, y_{i'})$ is exactly the residual of q at the original sample $(\Delta x_i, \Delta y_i)$. Therefore the objective value in Equation 10.11 is the same as the objective value in Equation 10.10 after the one-to-one change of variables induced by Equation 10.9.

Under the same full-rank assumption used in Equation 3.13 and Equation 3.14, the least-squares minimizer is unique. Since Equation 10.10 and Equation 10.11 are the same minimization problem written in two coordinate systems, their minimizers represent the same fitted polynomial. The only thing that changes is the basis in which that polynomial is written. Using that equivalence together with the derivative transformation in Equation 10.5, the rotated x' coefficient in the WVF fit is exactly the directional derivative already determined by $\hat{f}_x$ and $\hat{f}_y$.

The conclusion is therefore structural rather than numerical. For a fixed support and a complete polynomial basis, the rotated WVF fit does not create new first-order information. It rewrites the same fitted polynomial in rotated coordinates and then extracts the directional derivative already implied by the fitted gradient. If an implementation changes the sampled neighborhood with angle or introduces other anisotropic choices, then this exact equivariance argument no longer applies in the same form.

11 From the Wide View Filter to the Line Filter

The WVF defined in Section 8 evaluates one orientation-dependent directional stencil at one pixel. The Line Filter (LF) keeps the same WVF stencil at each candidate angle, but evaluates it at several shifted centers and combines the resulting directional responses [5]. The next four subsections define

those virtual centers, the WVF responses collected along the line, the Gaussian weighting applied to them, and the final orientation-selection rule.

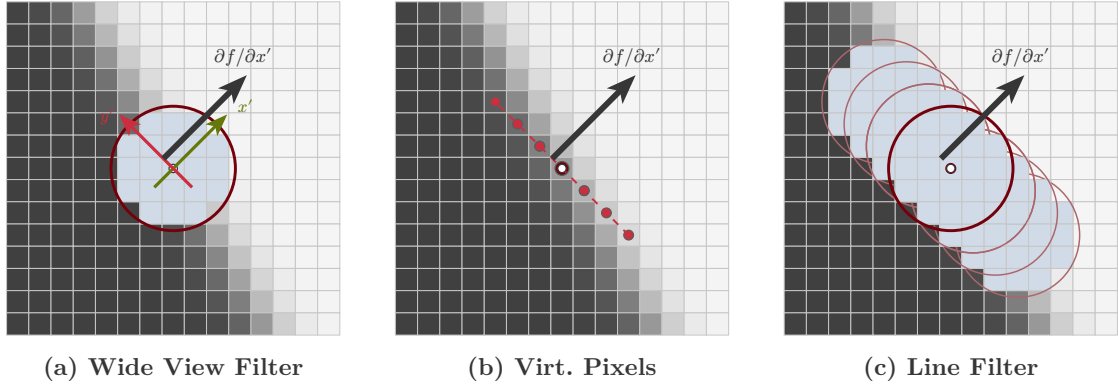


Figure 8: Schematic transition from the WVF to the LF. The WVF evaluates one circular point support at the target pixel for each candidate angle, while the LF places virtual evaluation points along a line and applies the same oriented point filter at each translated center before combining the resulting responses.

11.1 Virtual Evaluation Points Along the Candidate Direction

Using the sampled angle set from Equation 8.5, the LF first defines integer line offsets along the candidate tangent direction by

$$(\delta x_{j(\theta_k)}, \delta y_{j(\theta_k)}) = (\text{round}(-j \sin \theta_k), \text{round}(j \cos \theta_k)), \quad j \in \{-m, \dots, m\}. \quad (11.1)$$

Equation 11.1 shows that the parameter m controls the half-length of the virtual line. Since the rotated x' axis in Equation 8.1 and Equation 8.2 is the candidate normal direction, the offset in Equation 11.1 is perpendicular to that normal and therefore follows the candidate tangent. Using Equation 11.1, the j -th virtual evaluation point for the candidate angle θ_k is

$$(X_j, Y_j) = (X_0 + \delta x_{j(\theta_k)}, Y_0 + \delta y_{j(\theta_k)}). \quad (11.2)$$

Using the circular support from Equation 10.1, the sampled intensity vector gathered around the virtual point in Equation 11.2 is

$$\mathbf{b}^j(\theta_k) = [I(X_j + \Delta x_1, Y_j + \Delta y_1), \dots, I(X_j + \Delta x_{N_p}, Y_j + \Delta y_{N_p})]^\top. \quad (11.3)$$

Equation 11.3 is the first part inherited from the WVF. The support offsets $(\Delta x_i, \Delta y_i)$ are the same offsets used in Equation 10.1. The new LF element is that the same support is recentered at every virtual point indexed by j .

11.2 WVF Responses Along the Line

Using the WVF derivative row in Equation 8.4, the directional response at the j -th virtual point is

$$R_{j(\theta_k)} = \mathbf{p}_{f_{x'}}(\theta_k)^\top \mathbf{b}^j(\theta_k). \quad (11.4)$$

Equation 11.4 shows that each LF line sample is still a WVF response. The orientation-dependent stencil $\mathbf{p}_{f_{x'}}(\theta_k)^\top$ is inherited directly from Equation 8.4, while the virtual-point data vector $\mathbf{b}^j(\theta_k)$ is the shifted sample vector from Equation 11.3. Therefore the LF does not alter the underlying point-stencil construction. It evaluates that same WVF stencil repeatedly along the line indexed by j .

11.3 Gaussian-Weighted Line Response

The LF then combines the point responses from Equation 11.4 by a Gaussian-weighted average along the line. The line weights are

$$w_j = \frac{\exp\left(-\frac{j^2}{2\sigma_m^2}\right)}{\sum_{q=-m}^m} \exp\left(-\frac{q^2}{2\sigma_m^2}\right), \quad \sigma_m = \frac{m}{2}. \quad (11.5)$$

Using Equation 11.5 and Equation 11.4, the LF response at the candidate angle θ_k is

$$L(\theta_k) = \sum_{j=-m}^m w_j R_{j(\theta_k)}. \quad (11.6)$$

Equation 11.6 shows which part of the LF is new relative to the WVF. The WVF response in Equation 8.6 is one weighted sum over one support. The LF response in Equation 11.6 is a second weighted sum taken over the collection of WVF responses defined by Equation 11.4.

11.4 Selection of the Winning LF Direction

The LF uses the same discrete angle set from Equation 8.5 and the same absolute-response selection rule used for the WVF in Equation 8.7. The winning LF direction is therefore

$$\theta_*^{\text{LF}} = \arg \max_{\theta_k \in \Theta_{N_s}} |L(\theta_k)|. \quad (11.7)$$

Using Equation 11.7, the LF magnitude returned at that pixel is

$$G^{\text{LF}} = |L(\theta_*^{\text{LF}})|. \quad (11.8)$$

Equation 11.1 through Equation 11.8 complete the LF definition. The elements inherited from the WVF are the angle set in Equation 8.5 and the directional point stencil in Equation 8.4. The new LF elements are the shifted virtual points in Equation 11.2, the Gaussian line weights in Equation 11.5, and the line aggregation in Equation 11.6.

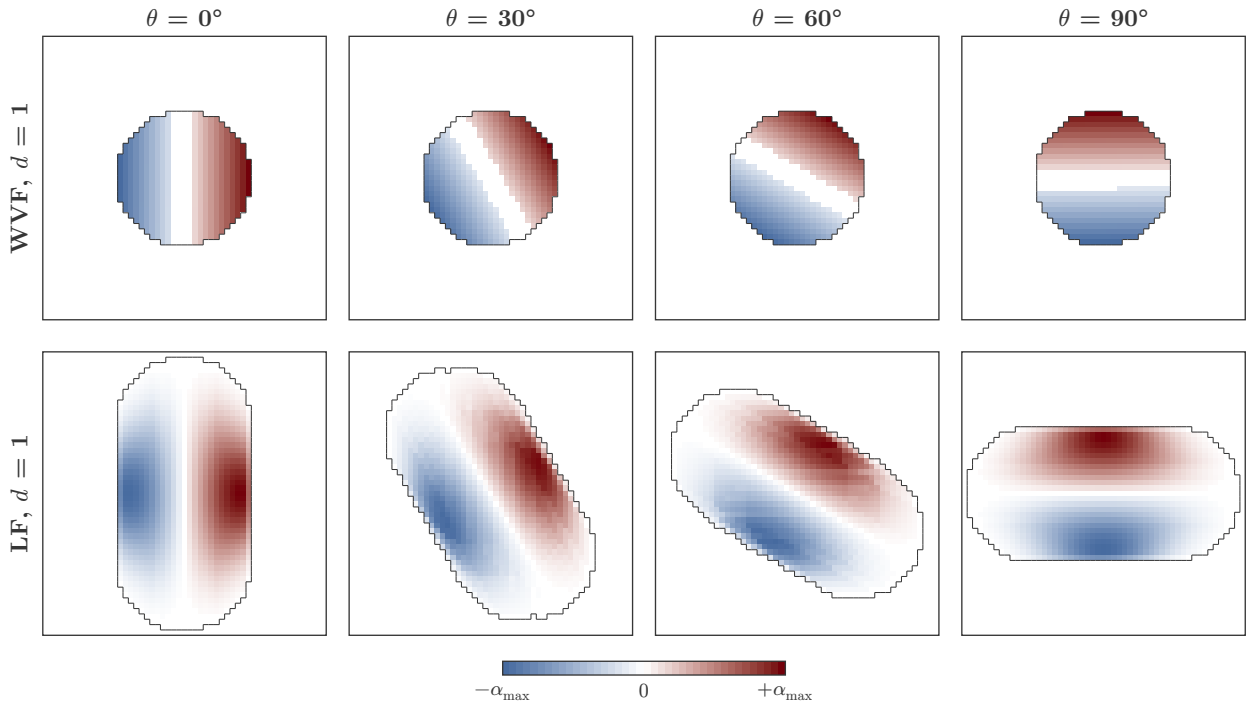


Figure 9: Comparison of the degree-1 WVF and LF stencils across four representative orientations. The WVF remains a compact circular point stencil, while the LF becomes an elongated anisotropic stencil because the WVF point response is accumulated over the translated line positions from Equation 11.2.

12 Collapsing the Line Filter into a Fused Anisotropic Stencil

The LF response in Equation 11.6 is a weighted sum of the WVF responses from Equation 11.4, and each WVF response is itself a weighted sum of sampled intensities. Because both stages are linear, the entire LF at a fixed angle can be rewritten as one weighted sum over image samples. The next subsections define the mathematics behind the fused anisotropic and show the fused anisotropic stencil that results from it.

12.1 Expanding the Line Response Into Nested Sums

Substituting the point response in Equation 11.4 into the line response in Equation 11.6 gives

$$L(\theta_k) = \sum_{j=-m}^m w_j \mathbf{p}_{f_{x'}}(\theta_k)^\top \mathbf{b}^j(\theta_k). \quad (12.1)$$

Using the sampled intensity vector in Equation 11.3, Equation 12.1 becomes

$$L(\theta_k) = \sum_{j=-m}^m \sum_{i=1}^{N_p} w_j p_{i(\theta_k)} I(X_j + \Delta x_i, Y_j + \Delta y_i), \quad (12.2)$$

where $p_{i(\theta_k)}$ denotes the i -th entry of the WVF derivative row $\mathbf{p}_{f_{x'}}(\theta_k)^\top$ from Equation 8.4. Using the virtual-point coordinates in Equation 11.2, the relative image offset addressed by the pair (j, i) is

$$\delta_{j,i}(\theta_k) = (\delta x_{j(\theta_k)} + \Delta x_i, \delta y_{j(\theta_k)} + \Delta y_i). \quad (12.3)$$

Using Equation 12.3, Equation 12.2 can be written relative to the target pixel as

$$L(\theta_k) = \sum_{j=-m}^m \sum_{i=1}^{N_p} w_j p_{i(\theta_k)} I(X_0 + \delta_{j,i}^x(\theta_k), Y_0 + \delta_{j,i}^y(\theta_k)). \quad (12.4)$$

Equation 12.4 is a simplified form of the expanded line response. It is a weighted sum over the raw collection of offsets defined by Equation 12.3, where each offset is addressed by one pair (j, i) and has an associated coefficient $w_j p_{i(\theta_k)}$.

12.2 Grouping Repeated Pixel Contributions

Equation 12.4 contains $(2m + 1)N_p$ index pairs (j, i) , but those pairs do not need to address distinct pixels. Using Equation 12.3, let

$$\tilde{S}_k = \{\tilde{\delta}_{\ell(\theta_k)} : \ell = 1, \dots, N'_k\} \quad (12.5)$$

be an enumeration of the distinct offsets appearing in the raw collection $\{\delta_{j,i}(\theta_k)\}$. Using Equation 12.5, Equation 12.4 can be regrouped by unique pixel location:

$$L(\theta_k) = \sum_{\ell=1}^{N'_k} \sum_{\{(j,i): \delta_{j,i}(\theta_k) = \tilde{\delta}_{\ell(\theta_k)}\}} w_j p_{i(\theta_k)} I(X_0 + \tilde{\delta}_{\ell}^x(\theta_k), Y_0 + \tilde{\delta}_{\ell}^y(\theta_k)). \quad (12.6)$$

Equation 12.6 changes only the order in which the terms from Equation 12.4 are collected. Each raw term appears in exactly one group, and the same coefficient is attached to that term in both forms. The only change is that the raw terms are now collected according to the unique pixel positions they address, rather than according to the index pairs that generate those positions.

12.3 The Fused Weight Map

Using the grouped form in Equation 12.6, define the fused weight assigned to the unique offset $\tilde{\delta}_{\ell(\theta_k)}$ by

$$\alpha_{k,\ell} = \sum_{\{(j,i):\delta_{j,i}(\theta_k)=\tilde{\delta}_{\ell}(\theta_k)\}} w_j p_{i(\theta_k)}. \quad (12.7)$$

Substituting Equation 12.7 into Equation 12.6 yields the fused LF response

$$L(\theta_k) = \sum_{\ell=1}^{N'_k} \alpha_{k,\ell} I\left(X_0 + \tilde{\delta}_{\ell}^{x(\theta_k)}, Y_0 + \tilde{\delta}_{\ell}^{y(\theta_k)}\right). \quad (12.8)$$

If the sampled intensities at those unique offsets are collected into the vector

$$\mathbf{g}_k = \left[I\left(X_0 + \tilde{\delta}_1^{x(\theta_k)}, Y_0 + \tilde{\delta}_1^{y(\theta_k)}\right), \dots, I\left(X_0 + \tilde{\delta}_{N'_k}^{x(\theta_k)}, Y_0 + \tilde{\delta}_{N'_k}^{y(\theta_k)}\right) \right]^{\top}, \quad (12.9)$$

then Equation 12.8 can be written compactly as

$$L(\theta_k) = \boldsymbol{\alpha}_k^{\top} \mathbf{g}_k. \quad (12.10)$$

Equation 12.10 is the *final* fused anisotropic stencil form of the LF at angle θ_k .

12.4 Weight Cancellation and Support Overlap

Using Equation 12.5, the fused support size satisfies

$$N'_k \leq (2m+1)N_p. \quad (12.11)$$

Equation 12.11 reflects the overlap created by the translated supports in Equation 11.2. When two or more pairs (j, i) map to the same unique offset in Equation 12.7, their weights are added before the image intensity is sampled. If those contributing terms have opposite signs, then the fused coefficient $\alpha_{k,\ell}$ in Equation 12.7 can be smaller in magnitude than the individual raw terms, and it can vanish when the signed contributions cancel exactly. This behavior follows from support overlap and coefficient aggregation in Equation 12.7. It is therefore a structural property of the fused stencil representation in Equation 12.8, not a change in the underlying LF defined by Equation 11.6.

13 Anisotropic Filter Families

The LF and the fused anisotropic stencil are two oriented constructions built from the WVF response. To compare them with other anisotropic families, it is helpful to place all of the filters in one common coordinate system and then state how each family chooses its support, its weighting law, and its derivative extraction rule.

13.1 A Common Oriented Coordinate System

Using the same rotation already introduced in Equation 8.1 and Equation 8.2, define the tangent-normal coordinates

$$u = -\Delta x \sin \theta + \Delta y \cos \theta, \quad v = \Delta x \cos \theta + \Delta y \sin \theta. \quad (13.1)$$

Equation 13.1 identifies u as the tangent coordinate and v as the normal coordinate. To keep the normal derivative coefficient explicit in the polynomial-fit families, define the ordered monomial basis

$$\psi_{d(u,v)} = \left[1, v, u, \frac{v^2}{2}, uv, \frac{u^2}{2}, \dots \right]^{\top}. \quad (13.2)$$

Using Equation 13.2, the second coefficient in the fitted parameter vector is always the first derivative in the normal direction v , which we are interested in.

13.2 Rectangular Anisotropic Polynomial-Fit Filter

Let $h_u > 0$ and $h_v > 0$ denote the tangent and normal half-widths of an oriented rectangle. Using the coordinates from Equation 13.1, the rectangular support is

$$S_{R(\theta)} = \{(\Delta x_i, \Delta y_i) \in \mathbb{Z}^2 : |u_{i(\theta)}| \leq h_u \text{ and } |v_{i(\theta)}| \leq h_v\}. \quad (13.3)$$

Using Equation 13.3 and the basis in Equation 13.2, the rectangular anisotropic least-squares fit is

$$\hat{\mathbf{z}}_{R(\theta)} = \mathbf{P}_{R(\theta)} \mathbf{b}_{R(\theta)}, \quad \mathbf{P}_{R(\theta)} = (\mathbf{A}_{R(\theta)}^\top \mathbf{A}_{R(\theta)})^{-1} \mathbf{A}_{R(\theta)}^\top, \quad (13.4)$$

where the rows of $\mathbf{A}_{R(\theta)}$ are $\psi_{d(u_{i(\theta)}, v_{i(\theta)})}^\top$ and $\mathbf{b}_{R(\theta)}$ contains the sampled intensities on $S_{R(\theta)}$. Since the second entry of Equation 13.2 is the normal derivative coefficient, the rectangular anisotropic derivative estimate is

$$\hat{f}_v^{R(\theta)} = \mathbf{p}_{f_v, R}(\theta)^\top \mathbf{b}_{R(\theta)}. \quad (13.5)$$

Equation 13.5 is the rectangular-support analogue of the circular derivative relation in Equation 5.5.

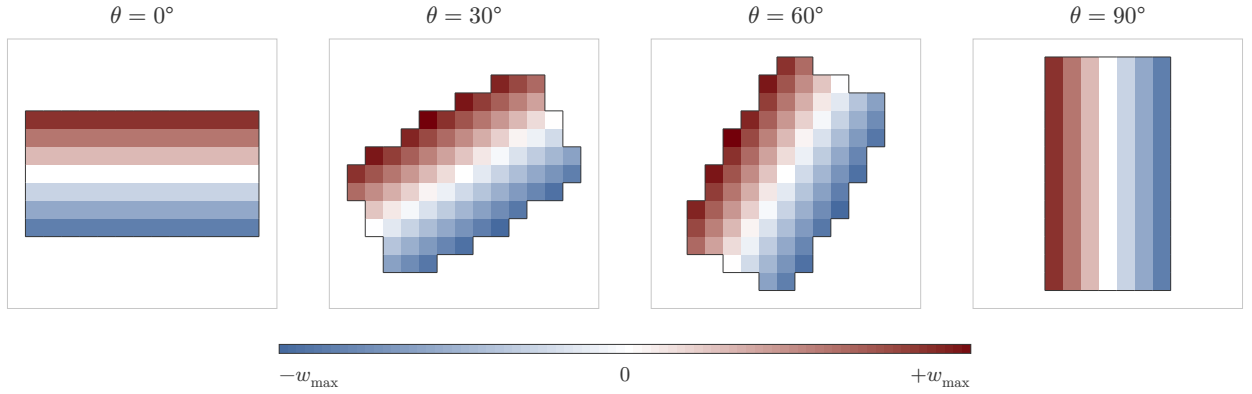


Figure 10: Orientation sweep for the degree-1 rectangular anisotropic polynomial-fit filter at $\theta = 0^\circ, 30^\circ, 60^\circ$, and 90° . In each panel, the elongated rectangular support remains aligned with the tangent coordinate u , while the extracted response is the derivative in the normal coordinate v from Equation 13.5.

13.3 Elliptical Anisotropic Polynomial-Fit Filter

Now let $a_u > 0$ and $a_v > 0$ denote the semi-axis lengths of an oriented ellipse measured in the same coordinates from Equation 13.1. The elliptical support is

$$S_{E(\theta)} = \left\{ (\Delta x_i, \Delta y_i) \in \mathbb{Z}^2 : \frac{u_{i(\theta)}^2}{a_u^2} + \frac{v_{i(\theta)}^2}{a_v^2} \leq 1 \right\}. \quad (13.6)$$

Using Equation 13.6 and the same basis in Equation 13.2, the elliptical anisotropic fit is

$$\hat{\mathbf{z}}_{E(\theta)} = \mathbf{P}_{E(\theta)} \mathbf{b}_{E(\theta)}, \quad \mathbf{P}_{E(\theta)} = (\mathbf{A}_{E(\theta)}^\top \mathbf{A}_{E(\theta)})^{-1} \mathbf{A}_{E(\theta)}^\top, \quad (13.7)$$

where the rows of $\mathbf{A}_{E(\theta)}$ are $\psi_{d(u_{i(\theta)}, v_{i(\theta)})}^\top$ for the samples in $S_{E(\theta)}$. Extracting the normal derivative coefficient gives

$$\hat{f}_v^{E(\theta)} = \mathbf{p}_{f_v, E}(\theta)^\top \mathbf{b}_{E(\theta)}. \quad (13.8)$$

Equation 13.5 and Equation 13.8 differ only through the support geometry in Equation 13.3 and Equation 13.6.

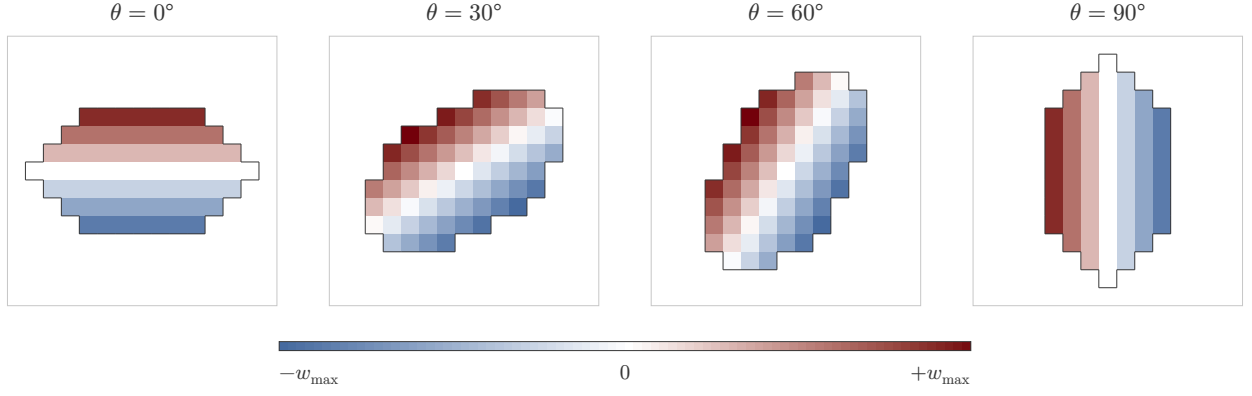


Figure 11: Orientation sweep for the degree-1 elliptical anisotropic polynomial-fit filter at $\theta = 0^\circ$, 30° , 60° , and 90° . The support is now elliptical rather than rectangular, but the derivative shown in each panel is still the normal derivative defined in Equation 13.8.

13.4 Anisotropic Gaussian Derivative Filter

The anisotropic Gaussian derivative family uses the same coordinates from Equation 13.1, but defines its weights analytically rather than through a polynomial fit. Its envelope is

$$G_{\sigma_u, \sigma_v}(u, v) = \exp\left(-\frac{1}{2}\left(\frac{u^2}{\sigma_u^2} + \frac{v^2}{\sigma_v^2}\right)\right). \quad (13.9)$$

Differentiating Equation 13.9 with respect to the normal coordinate v gives

$$K_{G(u,v)} = -\frac{v}{\sigma_v^2} \cdot G_{\sigma_u, \sigma_v}(u, v). \quad (13.10)$$

Using Equation 13.10, a truncated anisotropic Gaussian derivative response at one sampled angle θ_k can be written as

$$R_{G(\theta_k)} = \sum_{(\Delta x_i, \Delta y_i) \in S_{G(\theta_k)}} K_{G(u_i(\theta_k), v_i(\theta_k))} I_{X_0 + \Delta x_i, Y_0 + \Delta y_i}, \quad (13.11)$$

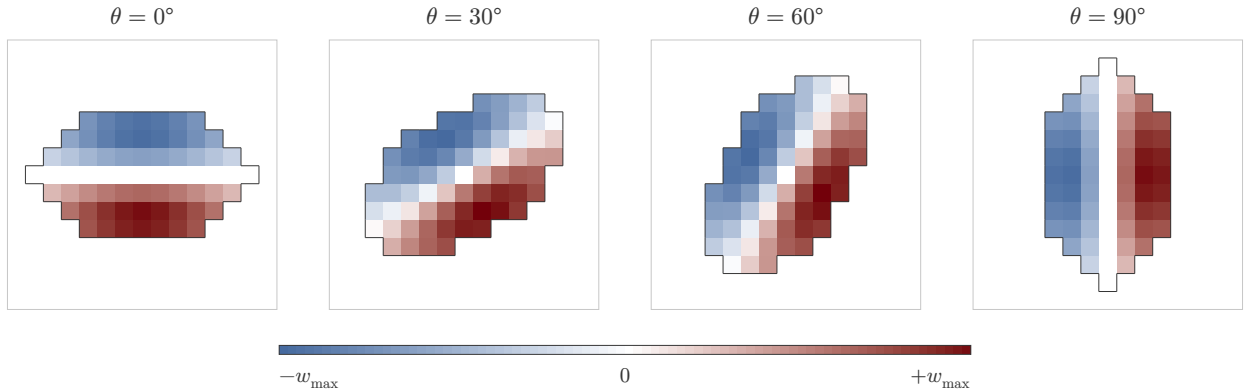


Figure 12: Orientation sweep for the anisotropic Gaussian derivative filter at $\theta = 0^\circ$, 30° , 60° , and 90° . The support remains elongated along the tangent coordinate u , while the positive and negative lobes track the normal derivative profile from Equation 13.10.

where $S_{G(\theta_k)}$ is the chosen finite truncation of the Gaussian support. Equation 13.5, Equation 13.8, and Equation 13.11 place the anisotropic polynomial-fit and anisotropic Gaussian families in one common response form.

14 Anisotropic Filters Relative to Isotropic Filters

The manuscript has now defined both isotropic and anisotropic filter families. The next step is to compare them structurally. This comparison does not rely on empirical results. It follows from the support geometry, the way orientation enters the response, and the parameter sets required to specify each family.

14.1 Support Symmetry

For any reference support $S(0)$, let the rotated support family be

$$S(\theta) = T_\theta S(0), \quad (14.1)$$

where T_θ denotes rotation by angle θ . Using Equation 14.1, a support is isotropic when it is invariant under that rotation:

$$T_\theta S = S \quad \text{for every } \theta. \quad (14.2)$$

Equation 14.2 is satisfied by the exact circular support in Equation 10.1 because its membership rule depends only on radius. By contrast, the families defined by Equation 13.3, Equation 13.6, and the LF construction from Equation 11.2 through Equation 12.8 are orientation-covariant families of the form Equation 14.1.

14.2 Directional Dependence and Angle Sampling

In the isotropic polynomial-fit case, the directional derivative at any continuous angle is already determined by the fitted gradient through Equation 10.5, and the maximizing continuous direction follows from Equation 10.8. The anisotropic families instead define one response for each sampled angle in Θ_{N_s} . Their generic selection rule can therefore be written as

$$G^{\text{aniso}} = \max_{\theta_k \in \Theta_{N_s}} |R(\theta_k)|. \quad (14.3)$$

Equation 14.3 differs from Equation 10.8 only in the domain over which the maximum is taken. The isotropic fit provides a continuous directional relation through Equation 10.5, whereas the anisotropic families are typically evaluated on the sampled set from Equation 8.5.

14.3 Locality and Line Aggregation

Using the oriented coordinates from Equation 13.1, define the tangent and normal half-spans of a support by

$$D_{u(S)} = \max_{(\Delta x, \Delta y) \in S} |u|, \quad D_{v(S)} = \max_{(\Delta x, \Delta y) \in S} |v|. \quad (14.4)$$

The corresponding aspect ratio is

$$\rho(S) = \frac{D_{u(S)}}{D_{v(S)}}. \quad (14.5)$$

Equation 14.4 and Equation 14.5 describe the geometric distinction between the families. The exact circular support in Equation 10.1 has $\rho(S_r)$ close to 1 in every direction. The LF inherits the same circular point support locally, but Equation 11.2 and Equation 12.8 extend the aggregate support tangentially so that the resulting fused support generally satisfies $\rho > 1$. The rectangular, elliptical, and anisotropic Gaussian families express the same geometric idea directly through the separate tangent and normal scales in Equation 13.3, Equation 13.6, and Equation 13.9.

14.4 Parameter Structure

The structural difference between isotropic and anisotropic families also appears in their parameter sets. Representative isotropic parameterizations are

$$\Pi_{\text{circ}} = \{N_p, d\}, \quad \Pi_{\text{sq}} = \{N, d\}, \quad \Pi_{\text{WVF}} = \{N_p, d, N_s\}, \quad (14.6)$$

while representative anisotropic parameterizations are

$$\Pi_{\text{LF}} = \{N_p, d, m, N_s\}, \quad \Pi_{\text{rect}} = \{h_u, h_v, d, N_s\}, \quad \Pi_{\text{ellip}} = \{a_u, a_v, d, N_s\}, \quad \Pi_{\text{G}} = \{\sigma_u, \sigma_v, N_s\} \quad (14.7)$$

Equation 14.6 and Equation 14.7 summarize the parameter structure used in the later computational and empirical comparisons. The isotropic polynomial-fit filters are determined primarily by support size and degree. The anisotropic families additionally encode orientation sampling and direction-dependent support shape or aggregation length.

15 Isotropic Polynomial-Fit Filters and the Gaussian Reference

The previous section compared isotropic and anisotropic families structurally. This section narrows the focus to the isotropic side and then places the isotropic polynomial-fit constructions beside the isotropic Gaussian derivative. The comparison remains mathematical and definitional. It establishes the reference models used in the later sections.

15.1 Circular Low-Degree Polynomial-Fit Filters

The circular-support construction in Section 5 gives the most rotation-symmetric polynomial-fit baseline in the manuscript. Its derivative rows are defined by Equation 5.5 and Equation 5.6, and the corresponding gradient magnitude and orientation are defined by Equation 5.8 and Equation 5.9. The low-degree results in Section 9 further show, through Equation 9.4 and Equation 9.17, that low polynomial degree provides exact affine recovery while keeping the derivative-row norm as small as the degree constraints permit on the chosen support.

Taken together, Equation 5.5, Equation 5.6, Equation 5.8, and Equation 9.17 identify the circular low-degree polynomial-fit filter as the primary isotropic baseline within the Savitzky–Golay family. It uses an area support, preserves the rotational symmetry expressed by Equation 14.2, and does not require a sampled-angle sweep once the paired derivative rows have been computed.

Equations Equation 10.1 and Equation 11.1 also define an isotropic envelope for the LF geometry. Every LF sample is obtained by taking one circular-support offset from Equation 10.1 and translating it by one of the line offsets from Equation 11.1. Therefore the farthest accessed lattice sites lie within the radius

$$R = r + m. \quad (15.1)$$

Using the radius-7 circular support from Equation 6.1 together with the representative LF half-width $m = 7$ gives $R = 14$. Figure Figure 13 shows the corresponding degree-1 circular Savitzky–Golay derivative pair on that enlarged disk. In this construction, the support is made large enough to contain every lattice point that the LF can reach while it rotates through its sampled angles.

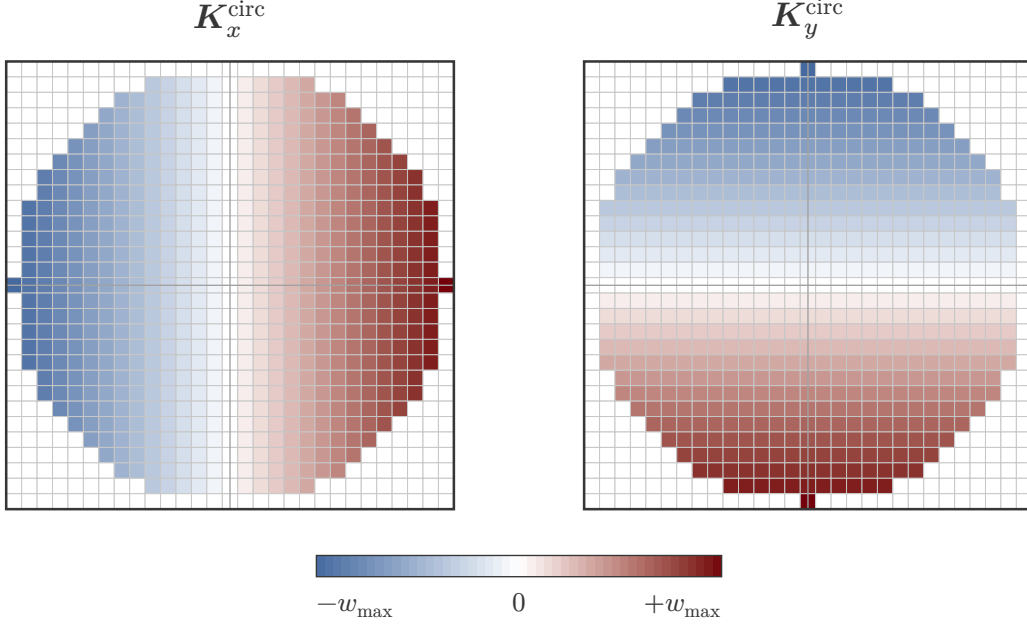


Figure 13: Degree-1 circular Savitzky–Golay derivative kernels on the enlarged isotropic disk of radius $R = r + m = 14$. With $r = 7$ and $m = 7$, this disk contains every lattice site that can be reached by the LF support as the line rotates through its sampled orientations. The left panel shows K_x^{circ} and the right panel shows K_y^{circ} on the shared 29×29 bounding box.

15.2 Square Low-Degree Polynomial-Fit Filters

The square-support construction from Section 3 remains a useful secondary isotropic baseline because its geometry and implementation are simple. Its derivative rows are defined by Equation 3.17 and Equation 3.18, and its combined responses are given by Equation 3.21 and Equation 3.22. Although the square support does not satisfy Equation 14.2 exactly, the low-degree arguments from Section 9 still apply because they follow from the exactness relation Equation 9.14 and the minimum-norm characterization Equation 9.15 rather than from circular symmetry alone.

The square filter is therefore a discrete axis-aligned reference rather than the primary isotropic target, while the circular low-degree filter remains the more rotation-symmetric member of the polynomial-fit family.

15.3 The Gaussian Derivative as a Reference Model

The isotropic Gaussian at scale σ is

$$G_{\sigma(x,y)} = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right). \quad (15.2)$$

Differentiating Equation 15.2 with respect to the coordinate axes gives the first-derivative kernels

$$\partial \frac{G_{\sigma}}{\partial} x = -\frac{x}{\sigma^2} \cdot G_{\sigma(x,y)}, \quad (15.3)$$

$$\partial \frac{G_{\sigma}}{\partial} y = -\frac{y}{\sigma^2} \cdot G_{\sigma(x,y)}. \quad (15.4)$$

Equation 15.2, Equation 15.3, and Equation 15.4 define a smooth isotropic gradient model whose scale is controlled by the single parameter σ . Like the isotropic polynomial-fit filters, the Gaussian derivative supplies paired first-derivative components and therefore supports the same continuous directional relation described by Equation 10.5 and Equation 10.8.

To place that analytic reference on the same spatial footprint as Figure Figure 13, we can sample Equation 15.3 and Equation 15.4 on the same radius- R disk defined by Equation 15.1. Figure Figure 14 shows that matched isotropic Gaussian pair for the broader representative choice $\sigma = \frac{R}{2}$, so that the visible Gaussian support occupies the same enlarged 29×29 footprint used by the large circular polynomial-fit filter.

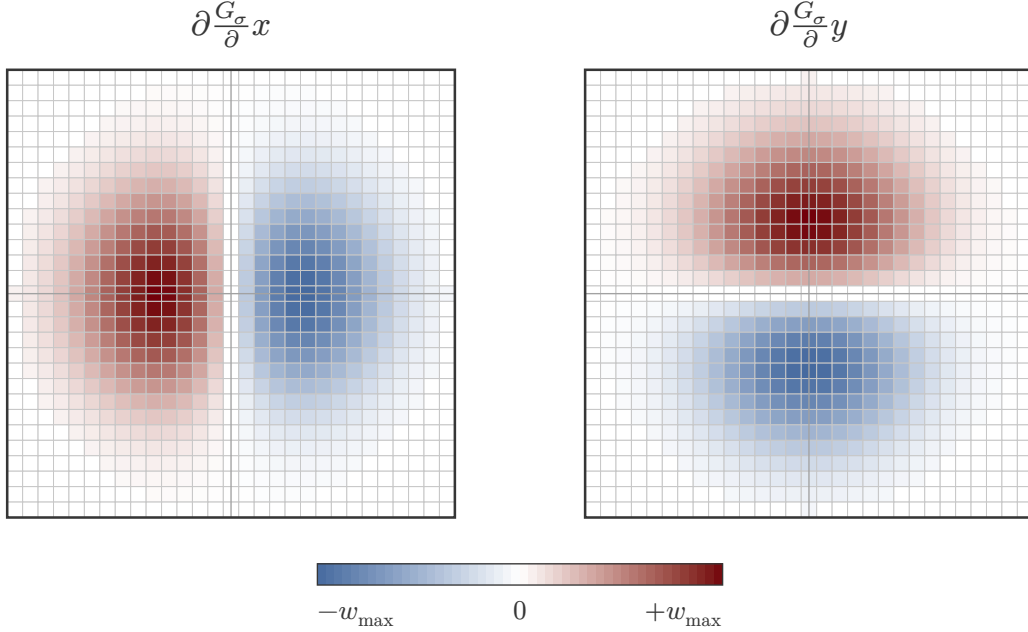


Figure 14: Isotropic Gaussian first-derivative pair sampled on the same enlarged radius- R disk used in Figure Figure 13. The left panel shows $\partial \frac{G_\sigma}{\partial} x$ and the right panel shows $\partial \frac{G_\sigma}{\partial} y$ for the broader representative choice $\sigma = \frac{R}{2}$.

15.4 The Isotropic Polynomial-Fit Filter Relative to the Gaussian Reference

The circular low-degree polynomial-fit filter and the isotropic Gaussian derivative address the same local first-derivative problem in different ways. The circular polynomial-fit filter derives its weights from the polynomial exactness conditions in Equation 9.14 and the minimum-norm characterization in Equation 9.15. The Gaussian derivative derives its weights from the analytic form of Equation 15.2 and its derivatives in Equation 15.3 and Equation 15.4.

The two constructions nevertheless share several structural properties. Both are isotropic in the sense of Equation 14.2. Both produce paired derivative components that can be combined into gradient magnitude and direction through relations of the form Equation 5.8 and Equation 10.6. Both also couple smoothing and differentiation into one linear operator. Figures Figure 13 and Figure 14 make that common structure visible on the same enlarged footprint. The difference is that the polynomial-fit filter is defined on a finite discrete support with degree parameter d , while the Gaussian reference is defined through the continuous scale parameter σ and then discretized for implementation.

16 Computational Structure and Cost

The theory sections above define several filter families with different support geometries and orientation-handling rules. This section compares their computational structure symbolically. The comparison uses the support sizes, angle counts, and fused-stencil sizes already introduced in the manuscript, and it focuses on the dominant gather-dot-product work required to apply each family at one pixel.

16.1 Operation Counts for the Filter Families

Let M_d denote the basis size from Equation 3.7. Since the pseudoinverse maps in Equation 3.14, Equation 8.3, Equation 13.4, and Equation 13.7 can be precomputed for a fixed geometry, the

runtime cost of applying a filter is determined by how many support samples are multiplied by weights and summed at each pixel. Representative per-pixel counts are

$$C_{\text{circ}} = 2N_p, \quad C_{\text{sq}} = 2N^2, \quad C_{\text{WVF}} = N_s N_p, \quad (16.1)$$

$$C_{\text{LF}} = N_s(2m+1)N_p, \quad C_{\text{fused}} = \sum_{k=0}^{N_s-1} N'_k, \quad (16.2)$$

and

$$C_{\text{rect}} = N_s N_R, \quad C_{\text{ellip}} = N_s N_E, \quad C_{\text{aG}} = N_s N_G, \quad (16.3)$$

where N_R , N_E , and N_G denote the active pixel counts of the rectangular, elliptical, and anisotropic Gaussian supports, respectively. Equation 16.1 through Equation 16.3 compare the dominant response-evaluation work only. Shared constant-time reductions, such as the final maximum over orientations, are lower-order terms relative to those counts.

To make Equation 16.1 through Equation 16.3 concrete, it is useful to evaluate them for one representative configuration. Using the exact radius-7 disk from Equation 6.1 gives $N_p = 149$. Using the 15×15 square support from Equation 4.1 gives $N = 15$. If the LF half-width is set to $m = 7$ and the sampled angle set is taken as the six orientations $0^\circ, 30^\circ, 60^\circ, 90^\circ, 120^\circ$, and 150° degrees, then Figure 15 and Table 1 summarize the corresponding dominant-work counts. The fused LF counts in that example come from the six enlarged stencil sizes $N'_k \in \{359, 349, 345, 359, 345, 349\}$, while the rectangular, elliptical, and anisotropic Gaussian counts are evaluated on matched anisotropic supports over the same six sampled orientations.

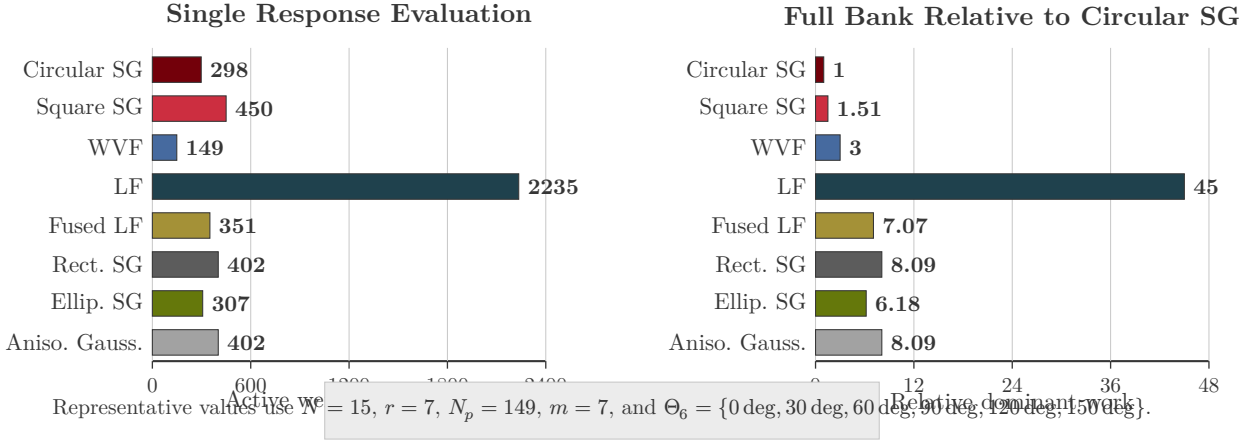


Figure 15: Representative dominant-work comparison for the filter families defined in Equation 16.1 through Equation 16.3. The left chart shows the active weighted samples used in one response evaluation. The right chart normalizes the full six-angle-bank cost by the circular low-degree polynomial-fit baseline.

Family	Formula	Representative calculation	Count
Circular SG	$2N_p$	$2 \cdot 149$	298
Square SG	$2N^2$	$2 \cdot 15^2$	450
WVF	$N_s N_p$	$6 \cdot 149$	894
LF	$N_s(2m+1)N_p$	$6 \cdot 15 \cdot 149$	13,410
Fused LF	$\sum_k N'_k$	$359 + 349 + 345 + 359 + 345 + 349$	2,106
Rectangular SG	$\sum_k N_{R,k}$	$435 + 389 + 387 + 421 + 389 + 389$	2,410
Elliptical SG	$\sum_k N_{E,k}$	$303 + 309 + 309 + 303 + 309 + 309$	1,842
Anisotropic Gauss- ian	$\sum_k N_{G,k}$	$435 + 389 + 387 + 421 + 389 + 389$	2,410

Table 1: Representative dominant-work calculations for the families in Equation 16.1 through Equation 16.3 with $N = 15$, $r = 7$, $N_p = 149$, $m = 7$, and the six sampled orientations 0° , 30° , 60° , 90° , 120° , and 150° degrees. The orientation-indexed anisotropic support counts are shown explicitly as six-angle sums.

16.2 Angle Sweeps and Aggregation Cost

The first structural distinction is whether the family depends on the sampled angle set Θ_{N_s} . The circular and square polynomial-fit filters form paired derivatives directly through Equation 5.5, Equation 5.6, Equation 3.17, and Equation 3.18, so their dominant work in Equation 16.1 is independent of N_s . The WVF, LF, fused stencil, rectangular anisotropic polynomial-fit, elliptical anisotropic polynomial-fit, and anisotropic Gaussian families all scale linearly with N_s , as shown by Equation 16.1, Equation 16.2, and Equation 16.3.

The second distinction is whether one candidate orientation requires one support evaluation or several. Equation 11.6 shows that the LF evaluates $(2m+1)$ point responses for each sampled angle, which is why Equation 16.2 contains the multiplicative factor $(2m+1)N_p$. Equation 12.10 removes the explicit repetition over j by replacing it with one fused stencil of size N'_k , but the angle sweep itself remains.

16.3 Cost Relative to Available First-Order Information

The WVF redundancy result in Section 10 is directly relevant to the costs above. Equation 10.5 shows that for an isotropic polynomial fit, the directional derivative at any angle is already determined by the paired first-derivative estimates $\hat{f}_x$ and $\hat{f}_y$. Equation 10.8 then shows that the maximizing continuous direction is obtained from those paired derivatives without a sampled-angle sweep. For that reason, the orientation-dependent families in Equation 16.1, Equation 16.2, and Equation 16.3 carry an angle factor that is absent from the paired isotropic baselines.

The LF and its fused form add a second layer of cost beyond that orientation factor. Equation 11.6 introduces the line half-width m , and Equation 12.7 shows that the fused stencil weights are generated by summing over the enlarged set of line-position and support-sample terms. The fused representation removes repeated runtime evaluation of those terms, but it still stores and applies one orientation-indexed stencil for each sampled angle. The computational distinction between the families is therefore structural. Some filters encode first-order information through one paired isotropic fit, while others encode it through a bank of orientation-indexed anisotropic stencils.

Bibliography

- [1] A. Savitzky and M. J. E. Golay, “Smoothing and Differentiation of Data by Simplified Least Squares Procedures,” *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964, doi: 10.1021/ac60214a047.
- [2] P. A. Gorry, “General Least-Squares Smoothing and Differentiation by the Convolution (Savitzky–Golay) Method,” *Analytical Chemistry*, vol. 62, no. 6, pp. 570–573, 1990, doi: 10.1021/ac00205a007.
- [3] P. Meer, E. S. Baugher, and A. Rosenfeld, “Frequency Domain Analysis and Synthesis of Image Pyramid Generating Kernels,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 11, pp. 1117–1139, 1991, doi: 10.1109/34.103270.
- [4] J. Luo, K. Ying, P. He, and J. Bai, “Properties of Savitzky–Golay Digital Differentiators,” *Digital Signal Processing*, vol. 15, no. 2, pp. 122–136, 2005, doi: 10.1016/j.dsp.2004.09.008.
- [5] H. Bagan and Y. Wang, “Line Filter: Edge Detection on the Basis of a Local Polynomial Model and Virtual Expansion Points,” *IEEE Access*, vol. 11, pp. 49684–49700, 2023, doi: 10.1109/ACCESS.2023.3277440.